

Migration der Endpoint-Management- Infrastruktur von Matrix42 Empirum auf die baramundi Management Suite (bMS)

*IT-Projektarbeit zur Erlangung des Berufsabschlusses
Fachinformatiker für **Systemintegration** (IHK Baden-Württemberg)*

Prüfling: Vladimir Leicht

Ausbildungsbetrieb: TASys GmbH

Praktikumsbetrieb: Landratsamt Enzkreis / Amt für IT und Digitalisierung

/ Pforzheim

Datum: Mai 2026

Inhaltsverzeichnis

1. Einleitung.....	1
1.1 Ausgangssituation.....	1
1.2 Analyse der bestehenden Empirum-Paketierung.....	1
1.3 Projektziel und Projektumfang.....	3
1.4 Wirtschaftliche Aspekte der Migration.....	4
2. Technische Voraussetzungen und Grundlagen.....	6
2.1 Bereitstellung der Server-VM.....	6
2.2 Administrative und netzwerkseitige Grundlagen.....	7
2.3 Installation und Grundkonfiguration des Servers.....	9
3. Installation der baramundi-Management Suite.....	11
3.1 Einrichtung der baramundi-Verwaltungskonten.....	11
3.2 Installation der Management Suite.....	15
3.2.1 Proxy-Verbindung für den bMS-Server.....	15
3.3 DIP-Installation und Konfiguration.....	17
3.4 Definition einer logischen Test-Client-Gruppe.....	19
3.5 Servermanagement und erste Job-Erstellung / - Zuweisung.....	21
3.5.1 Selbstmanagement des baramundi-Servers.....	23
3.6 PXE- und TFTP-Dienst.....	24
3.7 Vorbereitung des Windows-11-Deployments.....	25
3.7.1 Einbindung des Windows-11-Installationsmediums.....	25
3.8 Erstellung der WinPE-Bootumgebung.....	26
3.9 Erstellung eines Basis-Software-Bündels.....	28
3.9.1 Basis-Client-Software.....	28
3.10 Treiberintegration.....	30

4. Testrollout und Fehlerbehebung.....	32
4.1 Testrollout und Fehleranalyse.....	32
5. Projektabschluss.....	35
5.1 Soll-Ist-Vergleich.....	35
5.2 Zeitmanagement.....	36
5.3 Fazit und Ausblick.....	38
5.3.1 Fazit.....	38
5.3.2 Ausblick.....	38
5.4 Lessons Learned / Reflexionen.....	39
Anhang A: Begriffserläuterungen und Quellenangaben.....	42

1. Einleitung

1.1 Ausgangssituation

Die vorliegende Projektarbeit behandelt die Migration der bestehenden Endpoint-Management-Infrastruktur von Matrix42 Empirum^[1] auf die baramundi Management Suite^[2]. Der Projektdurchführungsort war das Landratsamt Enzkreis in Pforzheim, konkret das Amt für IT und Digitalisierung. Ziel war die Erarbeitung einer Pilotumgebung, die als Grundlage für die spätere dauerhafte Einführung eines stabileren Endpoint-Management-Systems dienen sollte. Ausgangspunkt ist die bisherige Verwaltung von Windows-Clients, Softwarepaketen, Treibern und Betriebssysteminstallationen über Empirum. Die Lösung erfüllt zwar grundsätzlich die Aufgaben der Clientverwaltung, hat sich im operativen Betrieb jedoch als wartungsintensiv, fehleranfällig und nur eingeschränkt skalierbar erwiesen.

1.2 Analyse der bestehenden Empirum-Paketierung

Ein wesentlicher Nachteil der bestehenden Umgebung liegt in der Paketierung und Bedienbarkeit. Die vorhandenen Empirum-Pakete basieren in der Praxis stark auf Setup.inf- und Empirum-Script-Strukturen. Am Beispiel des vorhandenen 7-Zip-Pakets wird sichtbar, dass selbst ein vergleichsweise einfaches MSI-Paket zahlreiche systemspezifische Abschnitte, Variablen, Registry-Prüfungen, MsiExec-Aufrufe, Logdatei-Auswertungen und eigene Fehlerbehandlungen enthält. Typische Konstrukte wie DoesRegKeyExist, DoesTextInFileExist, RebootRequired oder AbortMSIInst zeigen, dass die Paketlogik nicht nur aus einem Installationsparameter besteht, sondern in einer eigenen, proprietären und produktspezifischen Syntax gepflegt werden muss. Nachfolgend wird ein verkürztes Beispiel für ein Empirum-Installationsskript der Anwendung „7-Zip 24.01“ dargestellt; die Originallänge der Datei beträgt 203 Zeilen.

#####

[SetupInfo]

Author = Adm_zensiert ; Ersteller des Pakets

CreationDate = 13.02.2024 ; Datum der Paketerstellung

InventoryID = {B488FC15-B28F-4B63-A7D5-C9BD3707D009} ; Eindeutige Paket-ID

Description = Igor Pavlov 7-Zip 24.01 ; Paketbeschreibung

Method = MSI ; Installationsmethode: MSI

Tested on = ; Hier könnten getestete Systeme stehen

Dependencies = ; Abhängigkeiten^[14] zu anderen Paketen

Command line options = /S2 ; Zusätzliche Startparameter für das Setup

Last Change = 25.07.2023 ; Letzte Änderung am Paket

Build = 0 ; Build-Stand des Pakets

[VarDefInfo]

; <Variable>, <Type: 0=Computer 1=User>, <Allow NULL values: 0=No 1=Yes>

; Definition für benutzer- oder computerbezogene Variablen

·
·
·

[RepairMSI]

AddMeter -1

; Fortschrittsanzeige aktivieren

Call MsiExec /Fvomus "%SRC%\%VM_InstFile%" ALLUSERS="1" REBOOT=REALLYSUPPRESS
ARPSYSTEMCOMPONENT=1 /qb-! /Li "%MSILogFile%"

; MSI reparieren / alle wichtigen Komponenten prüfen / Log schreiben

If %ErrorLevel^[15]% == "3010" Then "RebootRequired" EndIf

; 3010 bedeutet Neustart erforderlich

If DoesTextInFileExist ("%ReInstSuccessMessage1031%", "%MSILogFile%") == "0" &
DoesTextInFileExist ("%ReInstSuccessMessage1031_1%", "%MSILogFile%") == "0" &
DoesTextInFileExist ("%ReInstSuccessMessage1033%", "%MSILogFile%") == "0" & DoesTextInFileExist
("%ReInstSuccessMessage1033_1%", "%MSILogFile%") == "0" Then "AbortMSIInst" EndIf

; Wenn kein Erfolgstext im Log steht, Installation als Fehler werten

Del "%MSILogFile%"

; Logdatei löschen

·
·
·

[InstallMSI]

AddMeter -1

; Fortschrittsanzeige aktivieren

```
Call MsiExec /I "%SRC%\%VM_InstFile%" ALLUSERS="1" REBOOT=REALLYSUPPRESS ARPSYSTEMCOMPONENT=1 /qb-! /Li  
"%MSILogFile%"
```

; Normale MSI-Installation im stillen Modus

```
If %ErrorLevel% == "3010" Then "RebootRequired" EndIf
```

; Neustartanforderung behandeln

```
#####
```

Der Usability-Nachteil ist offensichtlich. Änderungen an Softwarepaketen sind für Administratoren ohne tiefere *Empirum*-Erfahrung schwer nachvollziehbar, da Installationslogik, Rückgabewerte, Registry-Erkennung und Deinstallationsverhalten verteilt in mehreren Skriptabschnitten definiert werden. Für die im Landratsamt vorhandene Paketlandschaft bleibt jedoch maßgeblich, dass historisch gewachsene *Empirum*-Skripte verstanden und gepflegt werden müssen.

Hinzu kommt ein organisatorisches Risiko: Im laufenden Betrieb können derzeit nur zwei von fünfzehn Angestellten die bestehende Paketierung "sicher" erstellen, zuweisen und verteilen (wobei die Tätigkeit – aus zeitlichen Gründen - hauptsächlich nur von einem Angestellten durchgeführt wird). Abwesenheiten, Einarbeitungszeiten oder parallele Anforderungen wirken sich unmittelbar auf die Durchlaufzeit bei Softwarebereitstellungen und Client-Provisionierungen aus. Die technische Komplexität der Skripte verstärkt dieses Risiko, weil die operative Verantwortung nicht kurzfristig auf andere übertragen werden kann.

Neben der Paketierung bestehen weitere Einschränkungen im Betrieb. Die derzeitige PXE^[7]-basierte Betriebssysteminstallation ist störanfällig, das Patchmanagement für Drittanbieter-Software liefert nicht immer eine ausreichend transparente Rückmeldung. Diese Faktoren führen zu Nacharbeiten bei der Bereitstellung neuer Clients und erschweren ein verlässliches Lifecycle-Management.

1.3 Projektziel und Projektumfang

Ziel des Projekts ist die Einführung einer baramundi-Pilotumgebung als zentrale Unified-Endpoint-Management-Lösung. Im Gegensatz zu Matrix42 Empirum bietet baramundi eine intuitivere und leichter zugängliche GUI, was sich direkt in einer höheren Benutzerfreundlichkeit widerspiegelt. Der Projektrahmen umfasst den Aufbau eines bMS-Servers mit MS-SQL-Anbindung, die Konfiguration des Windows-11-Deployments über PXE, die Einbindung geeigneter Treiberprofile sowie die Bereitstellung ausgewählter Standardsoftware. Für den Testdurchlauf wurden vom Prüfling die Anwendungen „Citrix Workspace App“ (für den Zugriff auf virtualisierte Desktops und Anwendungen), „Cisco Webex“, „7-Zip (x64)“ (für Kommunikation, Meetings und Zusammenarbeit) und „Adobe Acrobat Reader (x64)“ sowie die dafür notwendigen Komponenten wie „Microsoft Visual C++ Redistributable“ und „Microsoft Edge WebView2 Runtime“ als „Basis-Client-Software“ ausgewählt. Diese Auswahl ermöglicht dem Benutzer das grundlegende Arbeiten und Kommunizieren an einem neu installierten Client. Durch die native Paketierung

über bDeploy^[5] soll die Bereitstellung nachvollziehbarer und wartbarer werden. Damit wird die Abhängigkeit von schwer wartbaren Altskripten reduziert und die Bedienbarkeit für weitere Administratoren verbessert.

Der Erfolg des Projekts wird anhand des Pilot-Rollouts auf Referenzclients bewertet. Maßgeblich sind ein funktionsfähiges Windows-11-Deployment, die erfolgreiche Installation der definierten Standardsoftware, eine nachvollziehbare Inventarisierung und eine stabilere Rückmeldung der Bereitstellungsvorgänge.

1.4 Wirtschaftliche Aspekte der Migration

Wirtschaftlich soll die neue Lösung, unter anderem administrative Durchlaufzeiten senken und damit die laufenden Betriebskosten reduzieren.

Öffentlich verfügbare Herstellerinformationen beschreiben die *baramundi Management Suite* als modular aufgebaute UEM-Plattform, die on-premises, hybrid oder gehostet betrieben werden kann. Die SaaS-Variante wird als pro Client abgerechnetes Modell mit übernommener Instandhaltung beschrieben. Für die Wirtschaftlichkeitsbetrachtung sind damit vor allem Laufzeitkosten, Wartungsanteile, Planbarkeit und Kapitalbindung relevant; die konkreten Beträge der Bewertung stammen aus den vorliegenden Nettoangeboten, die dem Prüfling vom zuständigen Sachbearbeiter des Betriebes zur Verfügung gestellt wurden.

Im Fünfjahresvergleich ist das 5-jährige Subscription-Modell mit 160.200,00 € netto die günstigste Variante. Danach folgen Kauf plus Wartung über fünf Jahre mit 175.715,00 €, Subscription über drei Jahre mit 187.620,00 €, Kauf plus Wartung über drei Jahre mit 200.480,00 €, Subscription über ein Jahr mit 221.280,00 € und Kauf plus Wartung über ein Jahr mit 231.715,00 €. Der Abstand zum günstigsten Kaufmodell beträgt 15.515,00 € netto. Für den definierten Projektzeitraum ist daher die fünfjährige Subscription wirtschaftlich vorzuziehen, während Kaufmodelle erst bei deutlich längerer Nutzung erneut betrachtet werden müssten.

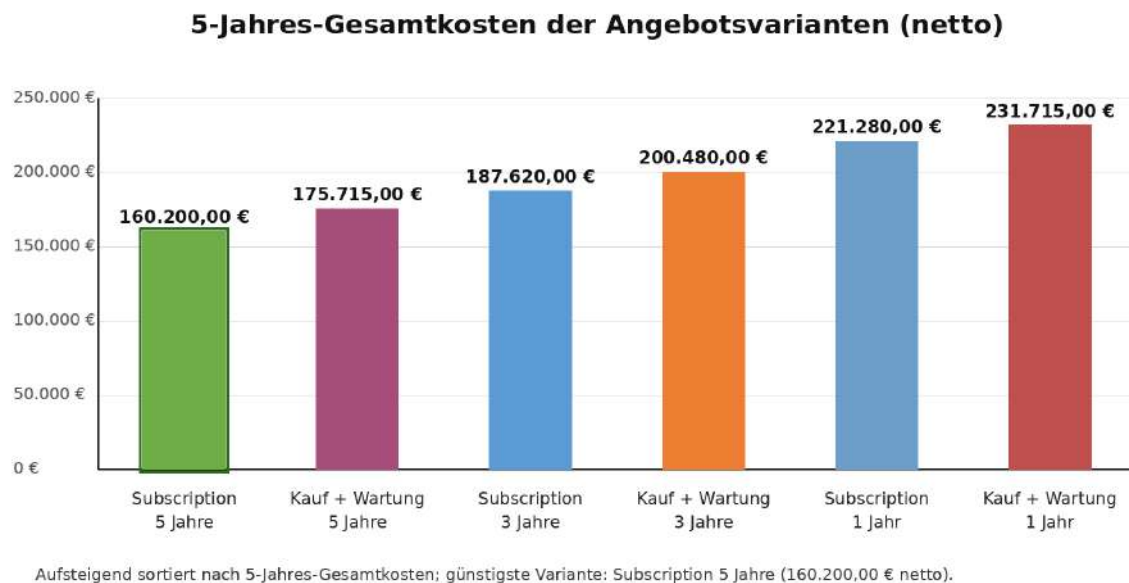


Diagramm 1: Kostenvergleich der Angebotsvarianten über fünf Jahre auf Nettobasis, aufsteigend nach Gesamtkosten

Leider können im Rahmen dieser Projektarbeit keine konkreten wirtschaftlichen Vergleichszahlen für Matrix42 Empirum vorgelegt werden. Der zuständige Mitarbeiter der IT-Abteilung, der seit April 2026 im Unternehmen tätig ist, verfügt selbst über keine verlässlichen Daten zu diesem Produkt. Da die Angebotspalette von Matrix42 (<https://marketplace.matrix42.com/de/?product-page=1>) sehr breit gefächert ist, hätte eine vollständige Erfassung aller im Betrieb genutzten Module sowie eine präzise Hochrechnung der Kosten – unter Berücksichtigung der verwalteten Endgeräte – den vorgegebenen Zeitrahmen dieser Projektarbeit bei Weitem gesprengt.

Bei rund 1.300 Endgeräten, die derzeit vom Amt für IT und Digitalisierung betreut und gewartet werden, sind Prozesskosten, Ausfallzeiten und der allgemeine Administrationsaufwand wirtschaftlich oft relevanter als die reine Lizenzpreisdifferenz. Eine funktionierende IT-Infrastruktur hat hierbei direkten Einfluss auf zahlreiche Mitarbeiter des Landratsamtes sowie indirekt auf die Bürger und Unternehmen des Landkreises. Der wirtschaftliche Aspekt der Lizenzkosten stellt daher nicht den primären Grund für die geplante Migration dar; vielmehr stehen Effizienz und Betriebssicherheit im Vordergrund.

Auf die Erstellung einer erweiterten Entscheidungsmatrix unter Einbeziehung alternativer Softwarelösungen wurde verzichtet. Die Entscheidung für die *baramundi Management Suite* stand im Vorfeld fest, da die zuständigen Administratoren bereits langjährige und positive Erfahrungswerte mit diesem Produkt vorweisen konnten. Durch den gezielten Rückgriff auf dieses bereits vorhandene Know-how entfallen zeitintensive Einarbeitungs- und Schulungsphasen vollständig, was die Projektlaufzeit verkürzt und das

Implementierungsrisiko erheblich minimiert.

2. Technische Voraussetzungen und Grundlagen

2.1 Bereitstellung der Server-VM

Die Bereitstellung des “baramundi-Management-Servers” erfolgte als virtuelle Maschine in der bestehenden “VMware-vSphere”-Umgebung des Landratsamts Enzkreis. Zunächst wurde die VM im vCenter angelegt, mit einem eindeutigen Hostnamen versehen und im vorgesehenen Ordner für Anwendungsserver einsortiert. Dadurch war der Server sauber in die vorhandene Struktur eingebunden und später eindeutig administrierbar.

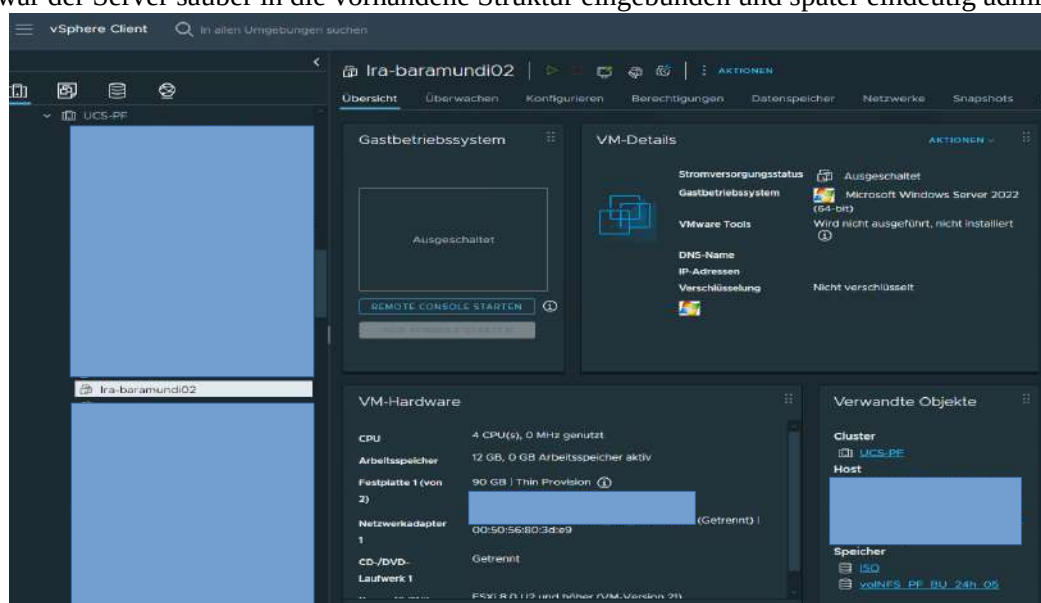


Abbildung 1: vSphere Client - Erstellung der virtuellen Maschine Ira-baramundi02

Die VM erhielt vier vCPUs, 12 GB RAM, eine 90 GB große Systemfestplatte sowie eine zusätzliche 500 GB große Datenfestplatte für das spätere DIP^[3]-Verzeichnis des baramundi-Servers zur Ablage von Softwarepaketen, Treibern und Betriebssystemimages. Die Trennung der Datenträger wurde bewusst gewählt, um Betriebssystem, Anwendungen und Repository-Daten logisch voneinander zu separieren und Ressourcenengpässe auf einem gemeinsamen Laufwerk zu vermeiden. Auch die Einbindung in das produktive Server-VLAN wurde bereits in diesem Schritt berücksichtigt, damit die Maschine von Beginn an im vorgesehenen Netzwerksegment betrieben werden konnte.

2.2 Administrative und netzwerkseitige Grundlagen

Für die spätere Administration des baramundi-Servers wurde im Active Directory eine eigene Sicherheitsgruppe angelegt. Diese Gruppe wurde nach dem AGDLP-Prinzip mit der lokalen

Administratorengruppe des Servers verknüpft. Dadurch werden administrative Rechte nicht direkt an einzelne Benutzerkonten vergeben, sondern zentral über die Domänenstruktur gesteuert. Die Berechtigungsvergabe bleibt damit nachvollziehbar, rollenbezogen und bei späteren personellen Änderungen einfacher anpassbar.

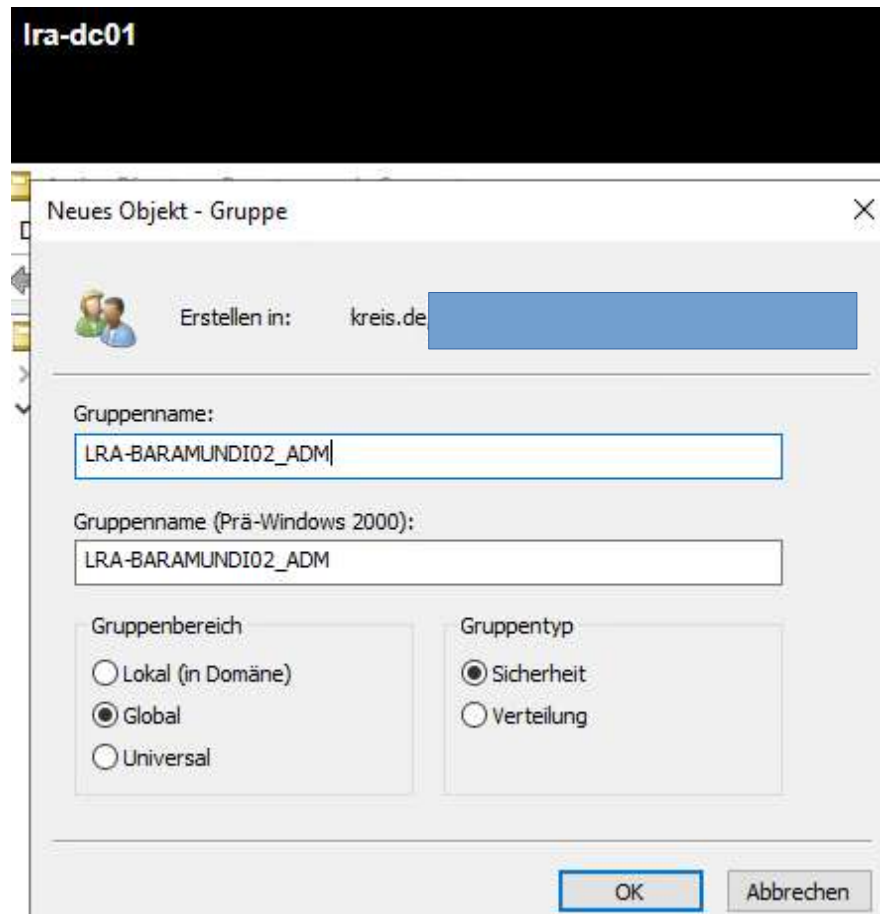


Abbildung 2: Erstellung der AD-Administrationsgruppe für den baramundi-Server

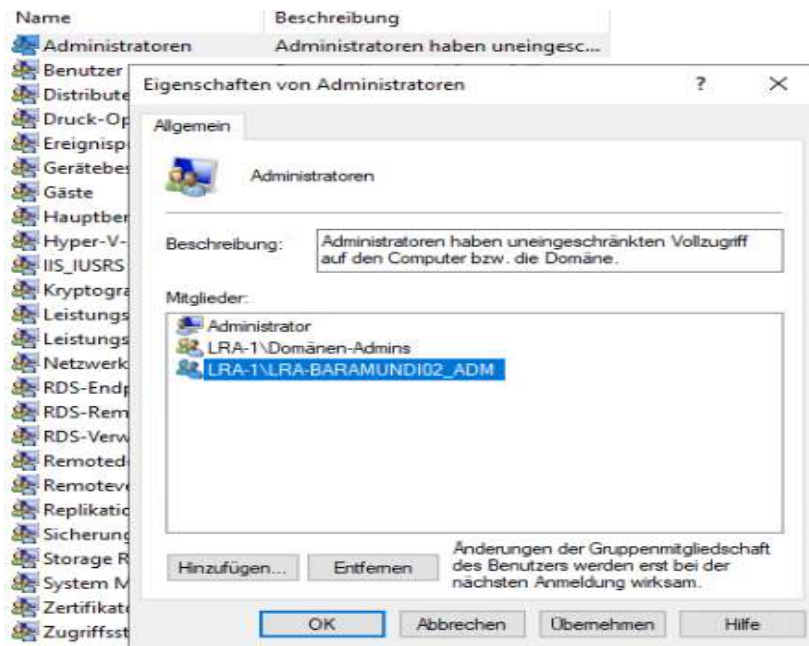


Abbildung 3: Windows Computerverwaltung - Schachtelung der neu erstellten AD-Sicherheitsgruppe in die lokale Gruppe Administratoren

Für den späteren PXE-Start der Referenzclients musste vor der Installation und weiteren Konfiguration des baramundi-Servers die DHCP-Konfiguration im vorgesehenen Testclient-Scope vorbereitet werden. Da die DHCP-Scopes Bestandteil der zentralen und vor allem komplexe und sensible Netzwerkinfrastruktur sind, erfolgte die technische Anpassung durch die zuständige Netzwerkadministration.

Im Scope wurden die für den Netzwerkstart erforderlichen Bootparameter hinterlegt: Option 66^[24] verweist auf den PXE-/TFTP-Server mit der Adresse x.x.133.33, Option 67 legt mit bootx64.efi^[25] den UEFI-Bootloader fest. Dadurch erhalten die Clients bereits beim DHCP-Vorgang die Information, von welchem Server und mit welcher Startdatei der PXE-Boot eingeleitet werden soll. Der baramundi-Server wurde hierfür in der DMZ im gleichen Segment wie die bereits bestehenden Server platziert.

Für die Pilotierung wurde an den vorgesehenen RJ45-Anschlüssen ein separates VLAN-Segment bereitgestellt (VLAN33), in dem sich zum späteren Zeitpunkt die Zielclients befinden würden. Die Testclients konnten dadurch gezielt mit diesen physischen Ports verbunden werden und erhielten über den konfigurierten DHCP-Dienst automatisch eine IP-Adresse aus dem dafür vorgesehenen Adressbereich. Die Zuordnung über definierte Anschlusspunkte erleichterte die Abgrenzung der Testumgebung vom produktiven Clientnetz – das während der Projektdurchführung weiterhin aktiv bleiben musste - und stellte sicher, dass DHCP-, PXE- und Deploymenttests nachvollziehbar auf die vorgesehenen Referenzgeräte beschränkt blieben.

Toggle Basic Mode

- General
- Member Assignment
- IPv4 DHCP Options
- VLAN Assignment
- IPv4 DDNS
- IPv4 BOOTP/PXE**
- IPv4 DHCP Thresholds
- Filters
- IF-MAP
- IPv4 IPAM Utilization Notification
- Extensible Attributes
- Permissions

Basic

PXE Lease Time
☐ Enable PXE Lease Time Seconds
Inherited from Grid Ira-dns

Deny BOOTP Requests
☐ Deny BOOTP Requests
Inherited from Grid Ira-dns

BOOTP Settings

Boot File
 Boot File

Next Server
 Next Server

Boot Server
 Boot Server

Abbildung 4: Infoblox IPAM/DHCP-Manager - Konfiguration der IPv4-DHCP-Bootparameter (Option 66: x.x.133.33) im Testclient-Scope

Toggle Basic Mode

- General
- Member Assignment
- IPv4 DHCP Options
- VLAN Assignment
- IPv4 DDNS
- IPv4 BOOTP/PXE
- IPv4 DHCP Thresholds
- Filters
- IF-MAP
- IPv4 IPAM Utilization Notification
- Extensible Attributes
- Permissions

Basic **Advanced**

Broadcast Address

Inherited from Grid Ira-dns

Custom DHCP Options

DHCP netbios-name-servers (44)
Inherited from Grid Ira-dns

DHCP netbios-node-type (46)
Inherited from Grid Ira-dns

DHCP

DHCP

Abbildung 5: Infoblox IPAM/DHCP-Manager - Konfiguration des UEFI-Bootloaders (Option 67: bootx64.efi)

Da Server und Zielclients in unterschiedlichen Netzwerksegmenten betrieben wurden, musste zusätzlich geprüft werden, dass die Weiterleitung der DHCP- und PXE-relevanten Anfragen routingseitig möglich ist. Diese Abstimmung ist für das OS-Deployment notwendig, weil ein korrekt vorbereitetes Boot-Image allein

nicht ausreicht, wenn die Clients den *PXE*-Dienst netzwerkseitig nicht erreichen. Nach der Anpassung war die Grundlage geschaffen, um den späteren *PXE*-Start der Referenzclients über baramundi nachvollziehbar zu testen.

2.3 Installation und Grundkonfiguration des Servers

Als Nächstes erfolgte die Installation von Windows Server 2022 Datacenter mit grafischer Oberfläche, da für die spätere Verwaltung der baramundi-Konsole ein lokaler GUI-Zugriff benötigt wurde. Im Anschluss wurde die IP-Konfiguration von DHCP auf eine feste Adresse umgestellt und die DNS-Auflösung auf die internen Server der Domäne ausgerichtet.

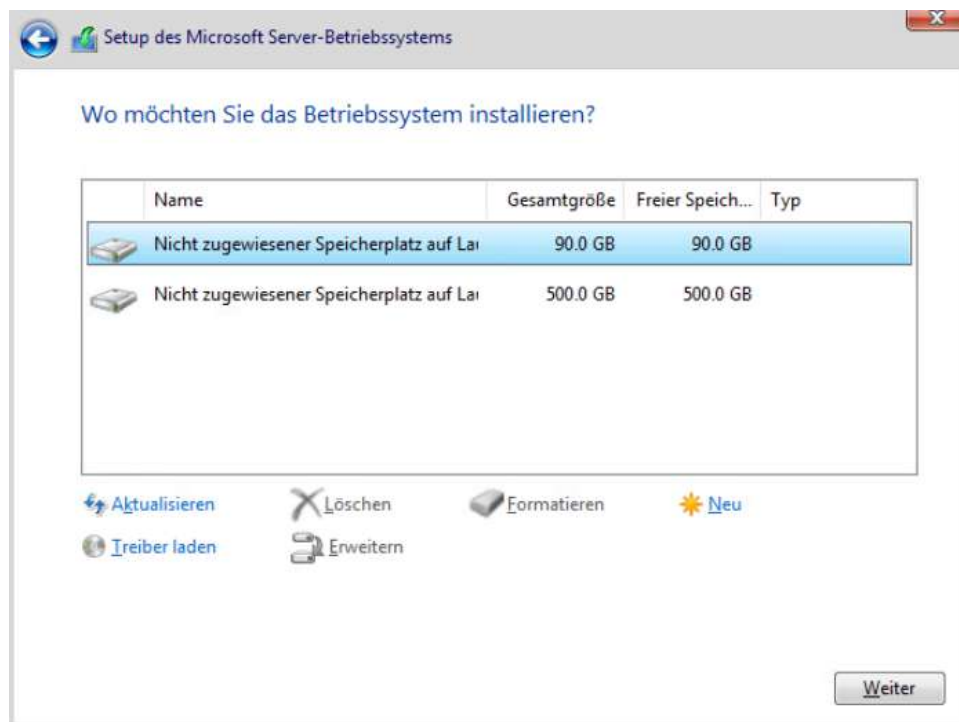


Abbildung 6: Installation des Windows-Server-2022-Systems auf der C-Partition der VM

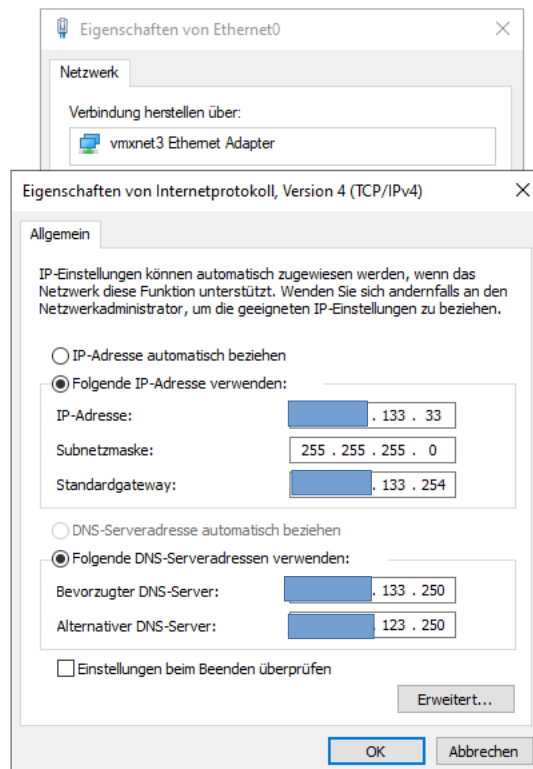


Abbildung 7: Statische IPv4-Konfiguration der virtuellen Netzwerkkarte (vmxnet3)

3. Installation der baramundi-Management Suite

Bevor die eigentliche baramundi Management Suite installiert werden konnte, mussten die erforderlichen Laufzeitumgebungen auf dem Server bereitgestellt werden. Dazu gehörten die Windows-Desktop-Runtime und die ASP.NET-Core-Runtime^[13] in der passenden 64-Bit-Version. Diese Komponenten sind notwendig, damit die grafische Konsole und die serverseitigen Dienste stabil betrieben werden können.

 windowsdesktop-runtime-8.0.26-win-x64	08.05.2026 09:27	Anwendung	57.084 KB
 aspnetcore-runtime-8.0.26-win-x64	08.05.2026 09:27	Anwendung	10.210 KB

Abbildung 8: Bereitstellung der Software-Abhängigkeiten (.NET Desktop Runtime und ASP.NET Core Runtime 64-Bit)

3.1 Einrichtung der baramundi-Verwaltungskonten

Vor der Datenbankeinrichtung und dem Start der eigentlichen Installation der *baramundi Management Suite* (*bMS*) mussten die benötigten Verwaltungs- und Dienstkonten zwingend im Active Directory vorbereitet werden. Ohne diese vorbereiteten Identitäten ist eine erfolgreiche Installation und der anschließende Betrieb des Systems technisch unmöglich. Die baramundi-Architektur basiert grundlegend darauf, dass verschiedene Hintergrundprozesse (wie der *bMS*-Serverdienst oder der *PXE*-Dienst) permanent administrative Aktionen im Netzwerk, auf den Ziel-Clients und innerhalb der SQL-Datenbank ausführen. Da diese Dienste ohne gültige Sicherheitskontexte weder starten noch kommunizieren können, ist die vorherige Bereitstellung dieser Konten eine kritische Systemvoraussetzung.

Die Verwendung dedizierter Konten erfüllt dabei zwei wesentliche Zwecke: Einerseits ermöglicht sie die technische Funktionsfähigkeit der Suite, andererseits stellt sie die Einhaltung des Prinzips der minimalen Rechtevergabe (Least Privilege) sicher. Serveraktionen, Clientzugriffe und Netzwerkzugriffe werden dadurch strikt voneinander getrennt und isoliert. Für das Projekt wurden die drei Kernkonten `baramundi_install`, `svc_baramundi` und `svc_baraNet` definiert.

Da die Erstellung von Domänenkonten und die Vergabe von Netzwerkberechtigungen in die Zuständigkeit der zentralen IT-Sicherheit fallen, wurden die Konten auftragsgemäß durch einen Verantwortlichen aus der Abteilung Infrastruktur angelegt und für die baramundi-Serverumgebung freigeschaltet.

1. **“baramundi_install”** (Server- und Infrastrukturkonto) wurde für Aufgaben vorgesehen, die direkt auf dem baramundi-Server ausgeführt werden. Dazu zählen unter anderem das Installieren von Agenten auf Windows-Geräten, das Ablegen heruntergeladener Dateien aus Download-Jobs, der Zugriff auf MSW- und Patch-Quellen^[16] auf dem primären DIP, das Bereitstellen von Daten für den Datenbankimport im FileImport-Verzeichnis, das Lesen von BDS-Dateien^[17], die AD-Synchronisation sowie das Anlegen von Computer-Konten im Rahmen von Betriebssysteminstallationen.
2. **“svc_baramundi”** (Installationsbenutzer) wurde als Installationsbenutzer für serverseitige Aktionen verwendet, die in einem definierten Benutzerkontext ausgeführt werden müssen. Das Konto führt diese Aktionen auf dem Server aus, greift für serverseitige Arbeitsschritte auf den DIP\$^[18] zu und kann bei entsprechenden Jobschritten für die Verbindung zu Windows-Clients verwendet werden.
3. **“svc_baraNet”** (Netzwerkbenutzer) wurde als Netzwerkbenutzer eingerichtet. Dieses Konto wird verwendet, wenn der baramundi Management Agent^[8] während einer Jobausführung auf Netzwerkressourcen zugreifen muss, beispielsweise bei Ausführungskontexten wie LocalSystem, lokalem Installationsbenutzer oder Installationsbenutzer. Außerdem dient es zur

Authentifizierung am baraDIP für bBT^[19]-Übertragungen und zur DIP-Synchronisation.

Anschließend wurde die Verbindung zur vorhandenen Microsoft-SQL-Server-Instanz eingerichtet. Die Datenbank sollte nicht lokal neu aufgebaut werden, sondern auf einer bereits vorhandenen, isolierten SQL-Umgebung laufen. Dazu wurde von einem Mitarbeiter der Infrastruktur-Abteilung ein dedizierter SQL-Login für baramundi auf dem SQL-Server eingerichtet.

Um die Sicherheit der isolierten SQL-Umgebung gemäß dem Least-Privilege-Prinzip zu gewährleisten, wurde der Account wie folgt konfiguriert:

Keine Server-Adminrechte: Der Login besitzt keine serverweiten Administrator-Rechte (sysadmin), und die Server-Rollen verbleiben auf dem Standardwert (public).

Berechtigung auf Datenbank-Ebene: Der Account erhält ausschließlich die Rolle *db_owner* für die spezifische baramundi-Datenbank.

Damit ist sichergestellt, dass der baramundi-Serverdienst (über die bCenter-Verbindung^[19]) im dauerhaften Betrieb alle notwendigen Lese-, Schreib- und Schema-Rechte innerhalb seiner eigenen Datenbank besitzt, ohne Zugriff auf andere Datenbanken oder die globale Serverkonfiguration zu erhalten.

The screenshot shows the 'baramundi Management Suite Datenbankmanager' window. The title bar includes the application name and standard window controls. Below the title bar, there is a subtitle 'Eine neue Datenbank erstellen'. A progress bar with five steps is visible: 'Verbinden' (1), 'Datenbank' (2), 'Domäne' (3), 'Sonstiges' (4), and 'Fertigstellen' (5). The 'Verbinden' step is currently active. Below the progress bar, there are several input fields: 'Datenbankserv' with the value '133.8', 'Integr. Authentifizierung verwenden' with an unchecked checkbox, 'Login-ID' with the value 'baramundi_pvr', 'Kennwort' with a masked password '*****' and a toggle icon, and 'Datenbank' with the value 'baramundi_02'. At the bottom right, there are two buttons: 'Zurück' and 'Weiter'.

Abbildung 9: baramundi Datenbankmanager - Schritt 1: Verbindungsaufbau zum MS SQL Server (x.x.133.8) mittels SQL-Authentifizierung

Die Datenbankverbindung wurde mit TLS abgesichert. Damit war die Backend-Kopplung technisch vollständig vorbereitet und konnte anschließend erfolgreich initialisiert werden.

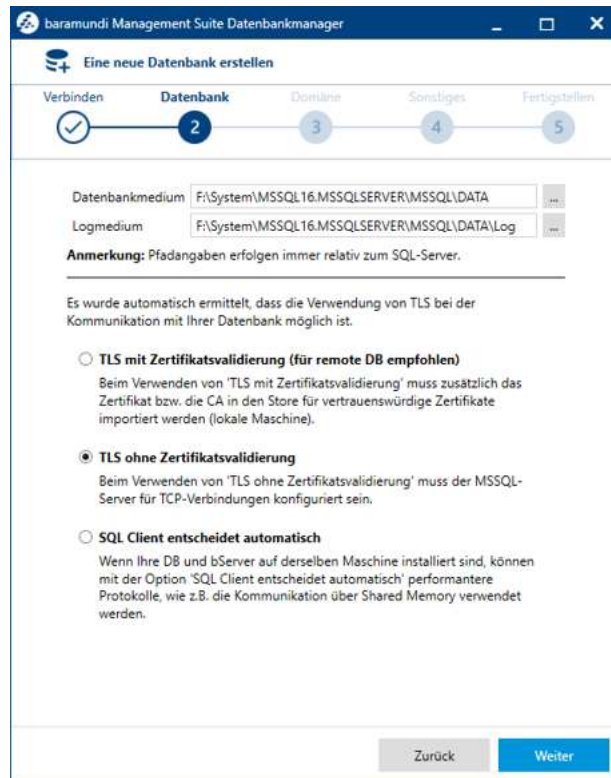


Abbildung 10: baramundi Datenbankmanager - Schritt 2: Spezifikation der Pfade für Primärdaten (.mdf) und Protokolle (.ldf) mit TLS-Verschlüsselung

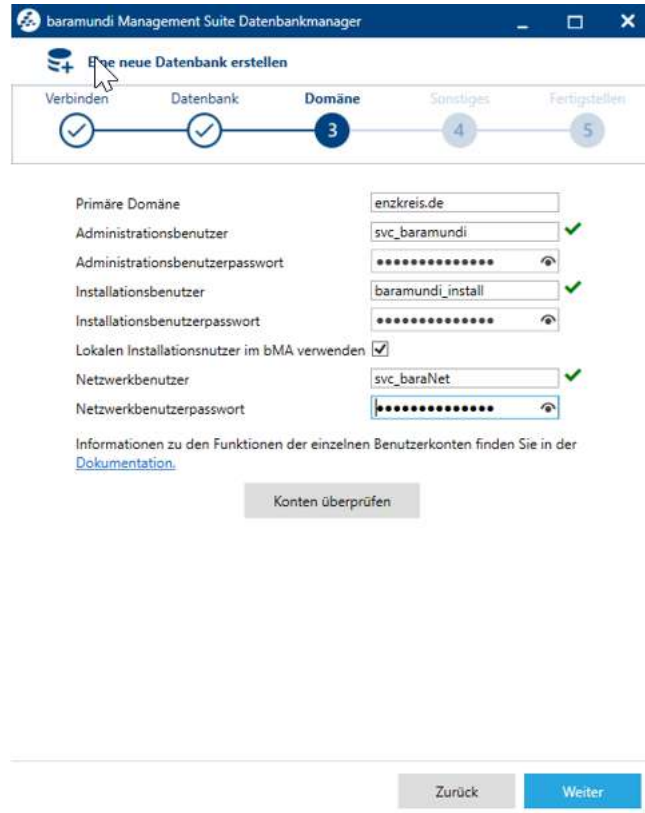


Abbildung 11: baramundi Datenbankmanager - Schritt 3: Integration und Validierung der AD-Dienstkonten

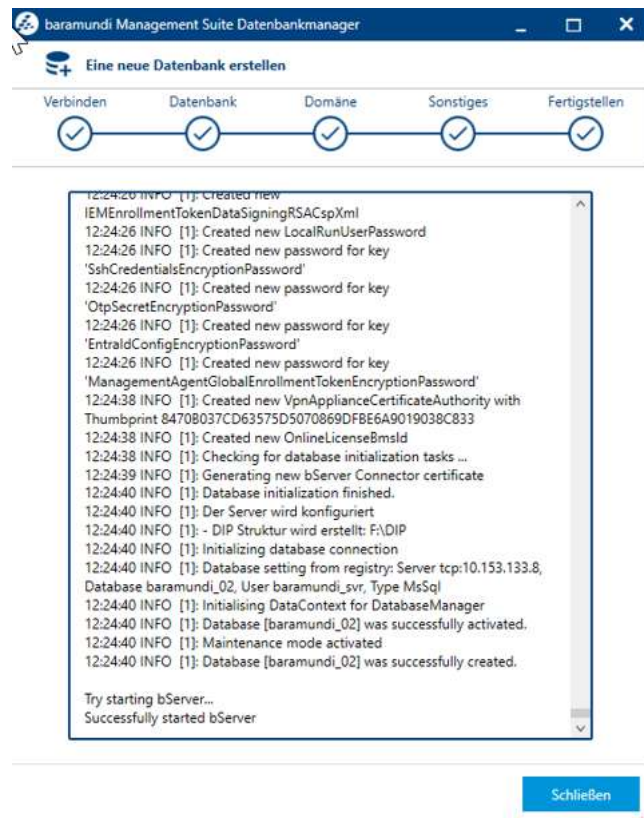


Abbildung 12: baramundi Datenbankmanager - Schritt 5: Schemaerstellung und Kaltstart des Hauptserverdienstes

3.2 Installation der Management Suite

Nach der erfolgreichen Vorbereitung von Server und Datenbank wurde die *baramundi Management Suite* 2025 R2 installiert.



Abbildung 13: Auswahl der zu installierenden Elemente der “baramundi Management Suite 2025 R2”

3.2.1 Proxy-Verbindung für den bMS-Server

Direkt nach der Installation konfigurierte der Prüfling die Proxy-Verbindung innerhalb der baramundi Suite. Diese Konfiguration ist notwendig, damit der baramundi Management Server über die zentrale Infrastruktur des Landratsamtes auf externe Update- und Paketquellen zugreifen kann. Für den operativen Betrieb ist diese Konfiguration erforderlich, um automatisierte Software-Updates und Definitions-Downloads über den dedizierten Proxy-Server dauerhaft sicherzustellen.

Die Hinterlegung der Proxy-Einstellungen ist für Administratoren ohne Dokumentation nicht intuitiv auffindbar, da die globale Internet-Anbindung nicht zentral in der Netzwerkverwaltung, sondern relativ tief verschachtelt unter “**Konfiguration>Server>Einstellungen>Downloader**” definiert wird.

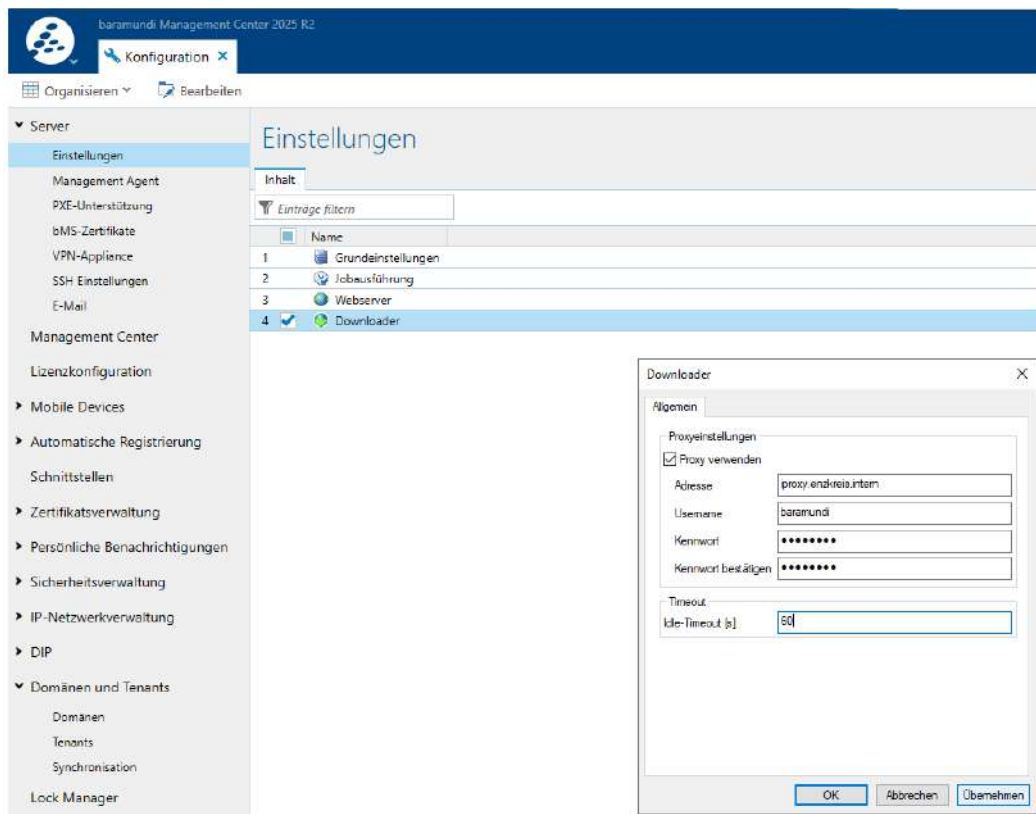


Abbildung 14: Konfiguration der Proxyverbindung mit den Zugangsdaten des “baramundi”-Kontos

Im Zuge der Installation wurde auch die Transportverschlüsselung zwischen Server und Clients aktiviert. Die Kommunikation läuft dabei über HTTPS auf Port 443, wodurch die Verbindung im internen Netz zusätzlich abgesichert wird. Nach dem Start der Dienste wurde die logische Struktur im Management Center eingerichtet, damit sich spätere Jobs, Gruppen und Zielsysteme sauber abbilden lassen.

Um eine sichere und verschlüsselte Kommunikation zwischen dem baramundi Management Server und externen Netzwerkkomponenten – wie dem baramundi Gateway für Homeoffice-Clients oder externen Drittsystemen (APIs) – zu gewährleisten, wurde die bConnect-Schnittstelle^[4] konfiguriert. Die Kommunikation wird hierbei über den Standard-HTTPS-Port 443 geleitet. Zur Absicherung der Zugriffe ist die integrierte Windows-Authentifizierung aktiv, um ausschließlich autorisierte Anfragen innerhalb der Domänenstruktur zu verarbeiten. Da es sich um eine reine On-Premises-Pilotierung handelt, wurden optionale Cloud-Connector-Dienste in dieser Phase bewusst deaktiviert gelassen.

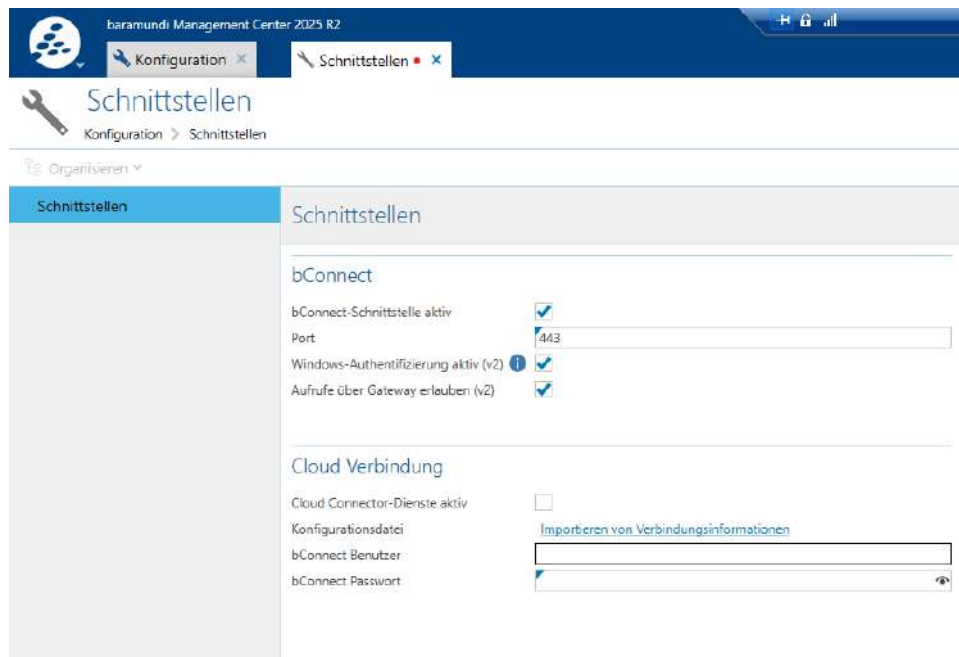


Abbildung 15: bMC Schnittstellen - Aktivierung der TLS-verschlüsselten bConnect-Schnittstelle auf Port 443

3.3 DIP-Installation und Konfiguration

Der *Distributed Installation Point (DIP)* war der nächste logische Installationspunkt innerhalb der baramundi-Management-Architektur. Er dient der effizienten Bereitstellung von Softwareverteilungen, Treibern und Betriebssysteminstallationen in verteilten IT-Umgebungen.

Da der *Distributed Installation Point* auf demselben Server wie die *baramundi Management Suite* betrieben wird, dient er in dieser Umgebung primär der Bereitstellung von Installationsquellen und Softwarepaketen. Die für verteilte Standorte typischen Vorteile eines *DIP*, wie die Entlastung von WAN-Verbindungen oder die lokale Bereitstellung von Daten in entfernten Netzwerksegmenten, kommen in dieser Testumgebung nicht zum Tragen. Die Implementierung erfolgte dennoch, da sie der von baramundi vorgesehenen Architektur entspricht und eine saubere sowie skalierbare Grundlage für eine spätere Erweiterung der Infrastruktur schafft.

Bei der Einrichtung dieser Komponente konnte der Prüfling zunächst keine Verbindung zwischen dem baramundi-Server und dem *Distributed Installation Point (DIP)* herstellen. Die Überprüfung der baramundi-seitigen Konfiguration sowie die Auswertung der verfügbaren Dokumentation führten zu keinem Ergebnis, sodass die Fehleranalyse auf die Server- und Netzwerkebene ausgeweitet wurde.

Nach einer vertieften Analyse sowie Rücksprache mit der betreuenden Fachkraft im Praktikumsbetrieb konnte der Prüfling die Ursache identifizieren. Diese lag in der Netzwerkkonfiguration des als virtuelle Maschine betriebenen Windows Server 2022. Im Dual-Stack-Betrieb wurden IPv6-Adressen gegenüber IPv4

standardmäßig priorisiert, während die bestehende baramundi-Infrastruktur überwiegend auf IPv4-Kommunikation ausgelegt war. Dies führte zu Problemen bei der Verbindung zum *DIP*.

Zur Behebung passte der Prüfling die Priorisierung der Netzwerkprotokolle an, sodass IPv4 bei der Adressauswahl bevorzugt wird, ohne IPv6 zu deaktivieren. Hierzu wurde in einer administrativen Eingabeaufforderung folgender Befehl ausgeführt:

```
netsh interface ipv6 set prefixpolicy ::ffff:0:0/96 60 4
```

Durch die Anpassung der Prefix Policy wurde die IPv4-Bevorzugung sichergestellt. Anschließend konnte der Prüfling die Verbindung zum *DIP* erfolgreich herstellen und die Einrichtung abschließen.

Im Rahmen der *DIP*-Einbindung mussten die Zugriffsrechte auf die entsprechende *DIP*-Freigabe eingerichtet werden. Hierfür wurden die relevanten administrativen Benutzer sowie die benötigten baramundi-Dienstkonten berechtigt. Die hierfür maßgeblichen Konten waren, wie bereits oben beschrieben, baramundi_install, svc_baramundi und svc_barasNet. Diese Konten erhielten Lese- und Schreibrechte auf das *DIP*-Verzeichnis.

Diese Berechtigungen sind erforderlich, damit der baramundi-Server, die Installationsdienste und der konfigurierte Netzwerkbenutzer Installationsdateien, Softwarepakete und Deployment-Daten im *DIP*-Verzeichnis ablegen, aktualisieren und auslesen kann. Ohne korrekt gesetzte Zugriffsrechte könnten Paketübertragungen, *DIP*-Synchronisationen oder spätere Software-Deployments nicht zuverlässig durchgeführt werden.

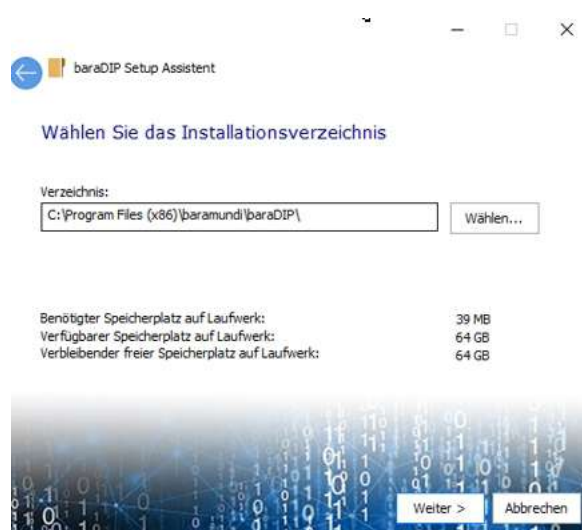


Abbildung 16: Installation des DIP-Moduls, das zur Verwaltung des später eingerichteten DIP-Servers (der auf der Partition F:\ platziert wurde) notwendig ist

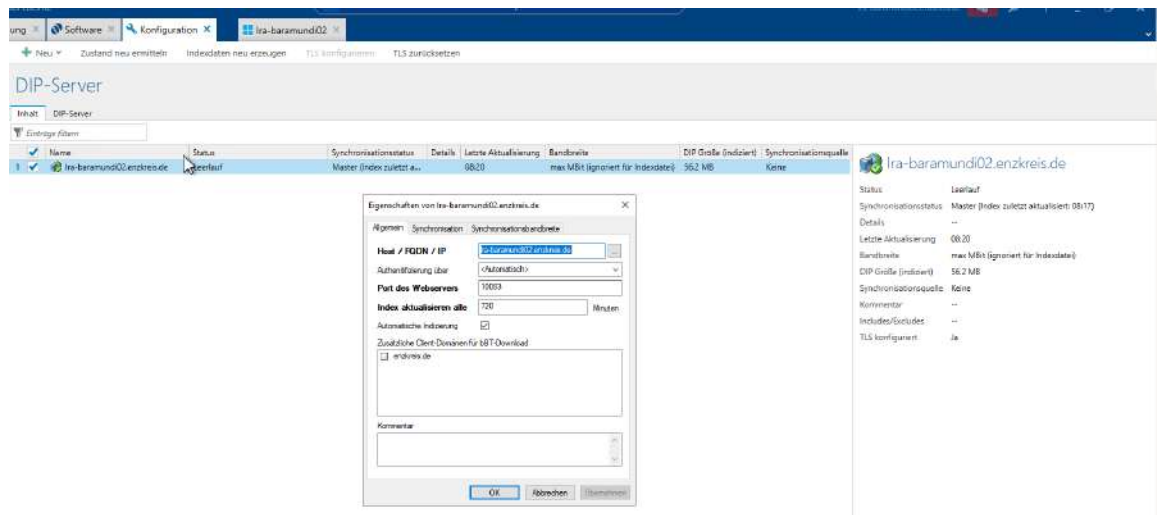


Abbildung 17: Erstellung und Konfiguration des DIP-Servers

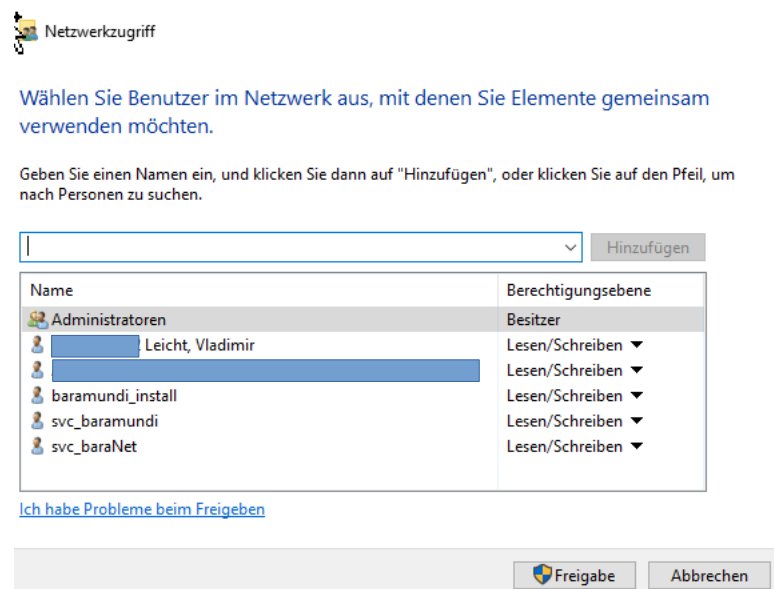


Abbildung 18: Rechtezuweisung der Konten für den Zugriff auf F:\DIP

3.4 Definition einer logischen Test-Client-Gruppe

Im nächsten Schritt wurde eine passende logische Gruppe sowohl für die zu verwaltenden Clients (**Testgruppe_UEM_Evaluation**), als auch für den baramundi-Server (**enzkreis.de**), erstellt. Zusätzlich wurde eine automatische Zuordnung neuer Clients in die "Testgruppe_UEM_Evaluation" konfiguriert. Sobald diese innerhalb des definierten Subnetz-Scopes erscheinen und per DHCP an den baramundi-Server weitergeleitet werden, verschiebt das Management Center die Clients automatisch in "Testgruppe_UEM_Evaluation" und initialisiert sowohl die Installation des Betriebssystems, als auch die des zusammengestellten Basissoftwares-Bundles ("Basis-Software-Clients").

Eigenschaften von enzkreis.de

Verfügbare Profile Unattended Parameter Berechtigungen

Allgemein Variablen Netzwerk Autoinstallation Bootclient Jobs

Name

DIP

lok. Adminpasswort setzen

Passwort zur Verifikation

Kommentar

OK Abbrechen Übernehmen

Abbildung 19: Erstellung einer logischen Gruppe “enzkreis.de” im Umgebungsbaum

Eigenschaften von Testgruppe_UEM_Evaluation

Verfügbare Profile Unattended Parameter Berechtigungen

Allgemein Variablen Netzwerk Autoinstallation Bootclient Jobs

Name

DIP

lok. Adminpasswort setzen

Passwort zur Verifikation

Kommentar

OK Abbrechen Übernehmen

Abbildung 20: Erstellung einer logischen Gruppe “Testgruppe_UEM_Evaluation” im Umgebungsbaum

3.5 Servermanagement und erste Job-Erstellung / -Zuweisung

Das best practice-Vorgehen sieht vor, dass auch auf den baramundi-Server ein Agent installiert wird. Dieses Vorgehen ermöglicht nicht nur eine Soft- und Hardwareinventur des Servers selbst, sondern auch ein Update-Management der auf dem Server installierten Software und der einzelnen baramundi-Module. Hierzu musste der Prüfling zunächst den baramundi-Server selbst, über die IP-Adresse, in die baramundi-Umgebung aufnehmen. Als logische Gruppe wurde hierfür die Gruppe “enzkreis.de” gewählt, damit eine saubere Trennung zu den späteren Clients, die in die logische Gruppe “Testgruppe_UEM_Evaluation” erfasst werden sollten, gegeben war. Nachdem dies erledigt war, konnte ein Agenten auf dem baramundi-Server ausgerollt und einen **baramundi-Job** mit der Bezeichnung **Inventur_HW_SW_WMI** erstellt werden, der auch als Template für zukünftige Inventur-Job-Zuweisungen an Clients verwendet werden konnte.

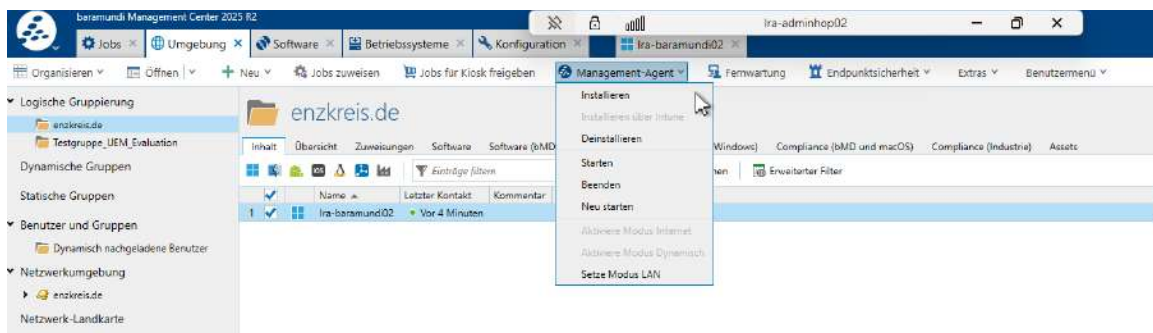


Abbildung 21: Logische Gruppierung des baramundi-Servers und Agentsteuerung im Management Center

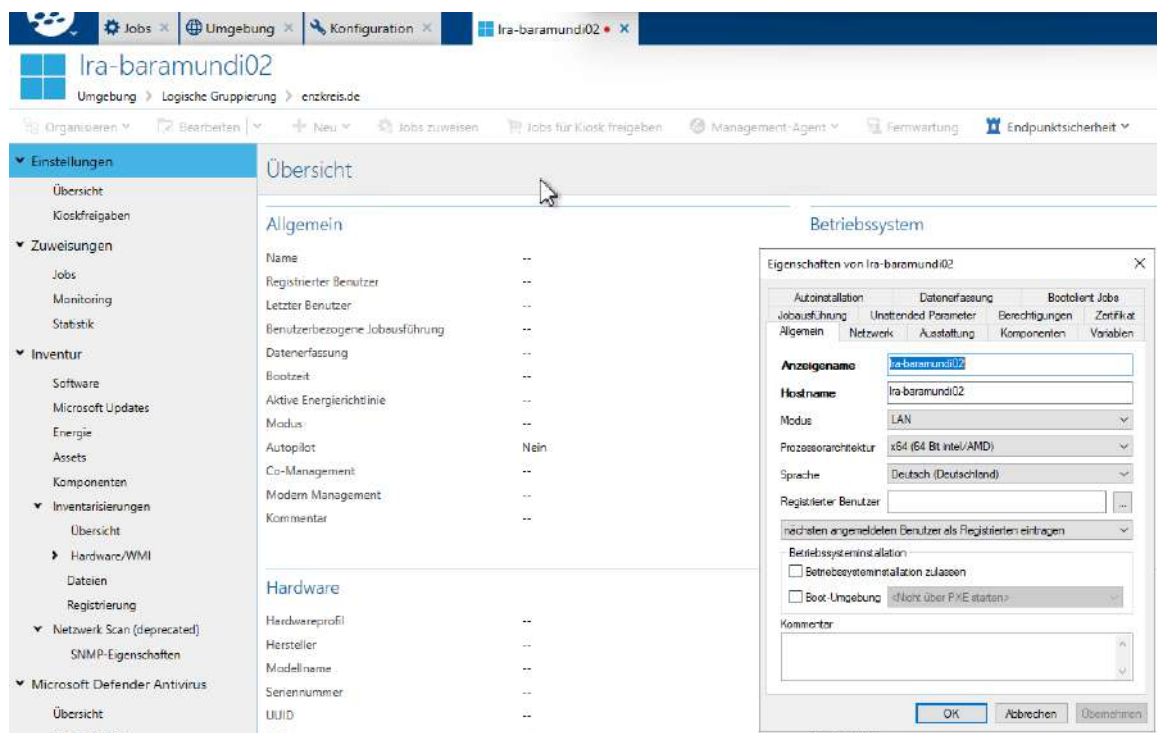


Abbildung 22: Einbindung des baramundi-Servers in die Management Suite-Umgebung

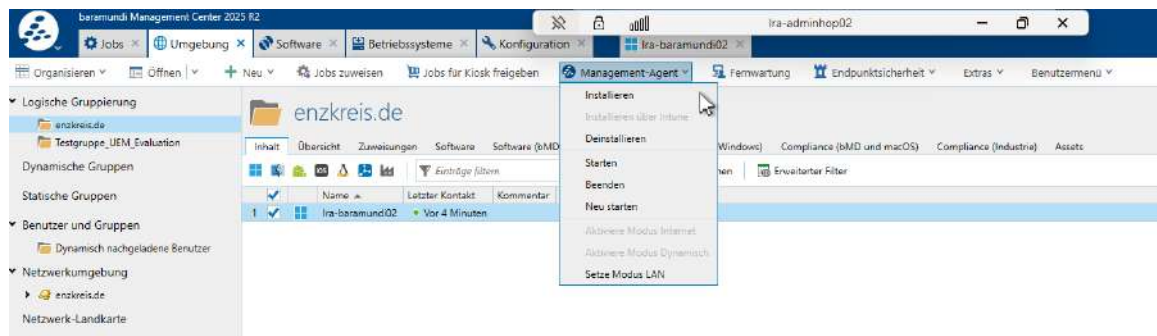


Abbildung 23: bMC – Installation des Agents auf dem baramundi-Server zur automatischen Systempflege und Inventur

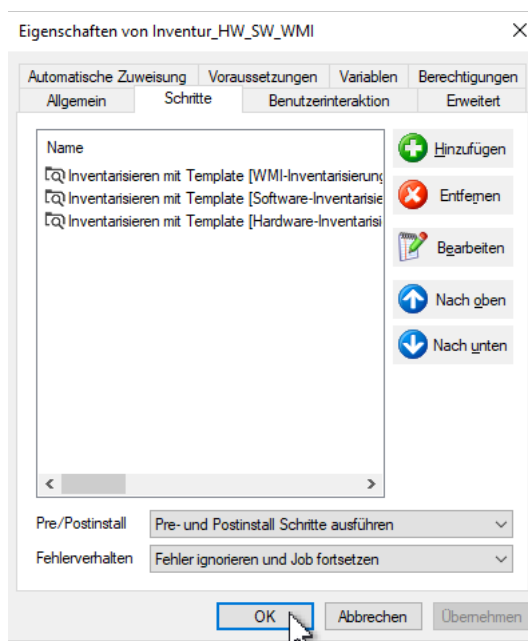


Abbildung 24: Erstellung eines Inventur-Jobs

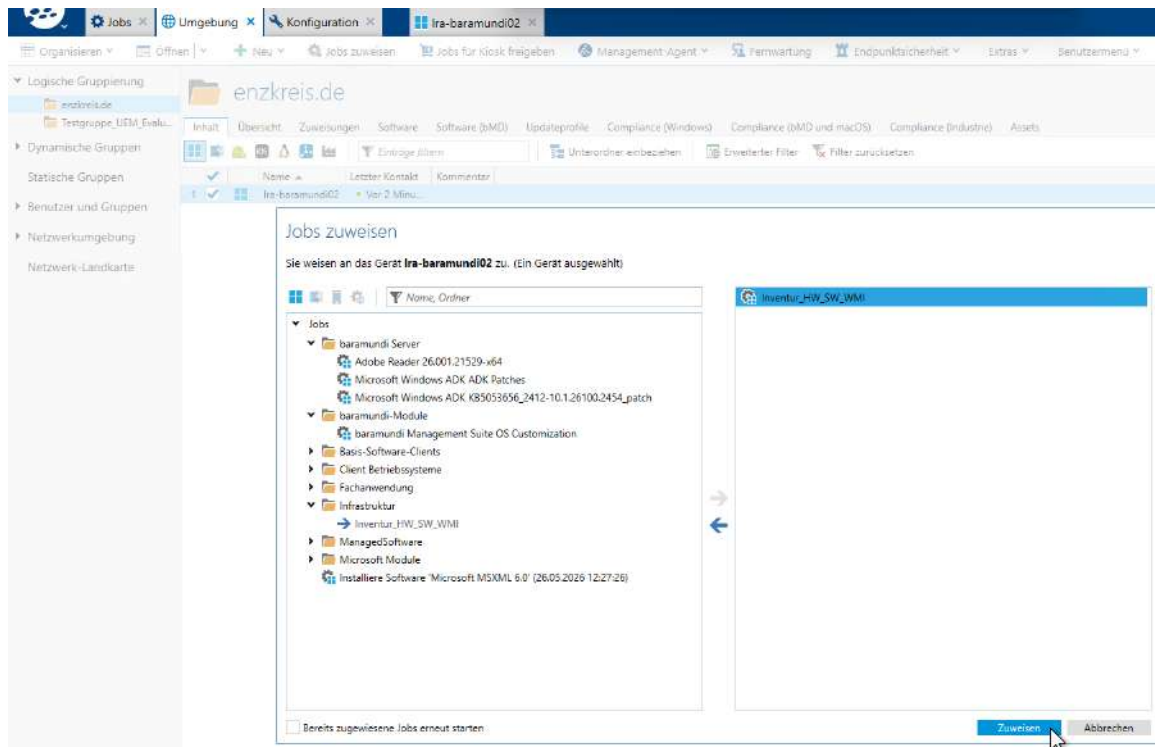


Abbildung 25: Zuweisung des Inventur-Jobs zum baramundi-Server

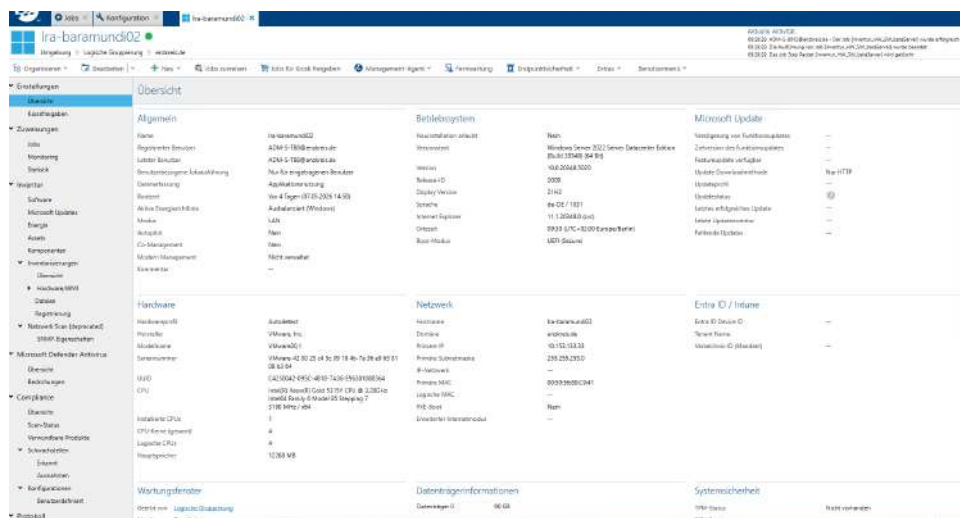


Abbildung 26: Registerkarte Hardware-Inventur: Erste erfolgreiche Auslesung der physischen Komponenten

3.5.1 Selbstmanagement des baramundi-Servers

Zur Systempflege des baramundi-Servers wurde ergänzend ein Infrastruktur-Job für das Selbstmanagement eingerichtet. Der Job kombiniert Software-Inventarisierung, Windows- und Microsoft-Updates sowie die Aktualisierung der Microsoft-Defender-Definitionen. Damit kann der Management-Server selbst über die

Suite überwacht und grundlegend aktuell gehalten werden, ohne die eigentlichen Client-Rollout-Jobs zu vermischen. Der Screenshot zeigt den erfolgreich beendeten Inventarisierungsjob sowie die vorgesehenen Jobschritte für Update- und Definitionspflege.

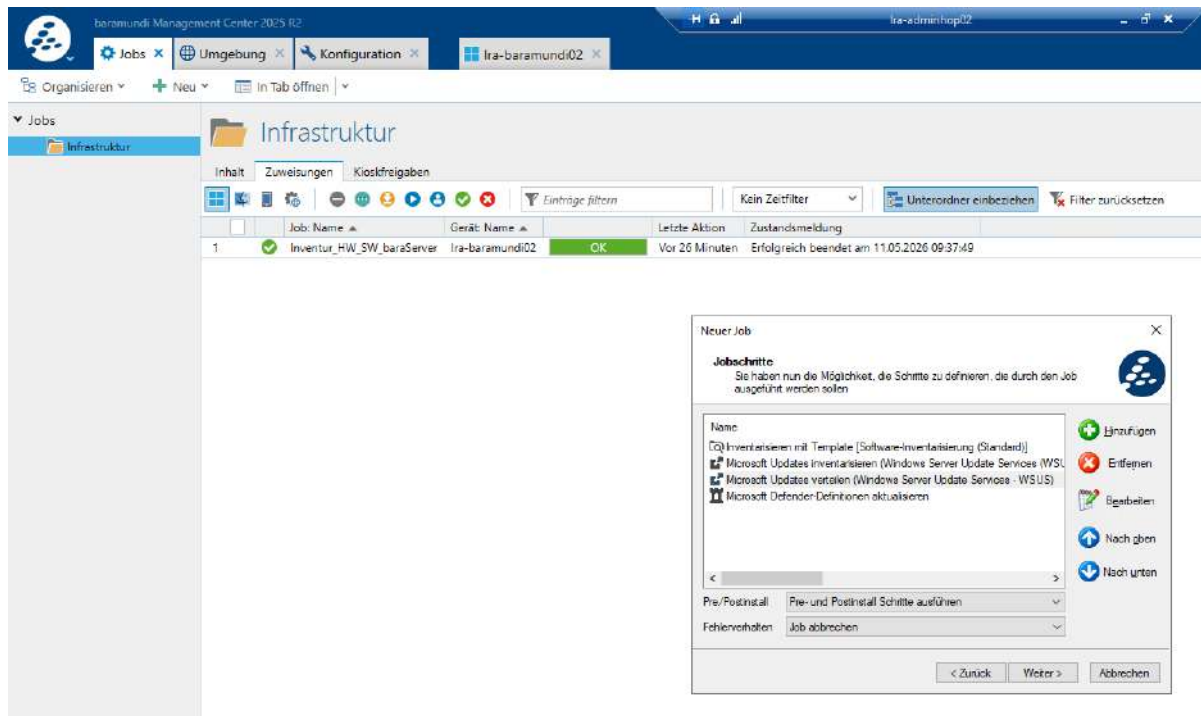


Abbildung 27: Selbstmanagement-Job des baramundi-Servers mit Inventarisierung und Microsoft-Updates

3.6 PXE- und TFTP-Dienst

Für das automatisierte Betriebssystem-Deployment wurde die *PXE*-Infrastruktur vorbereitet. Dazu gehörte die Aktivierung und Konfiguration des baramundi *PXE*- und *TFTP*-Dienstes. Der *TFTP*-Dienst stellt die Startdateien bereit, während die eigentliche *PXE*-Steuerung in der bestehenden DHCP-/Netzwerkconfiguration berücksichtigt wurde.

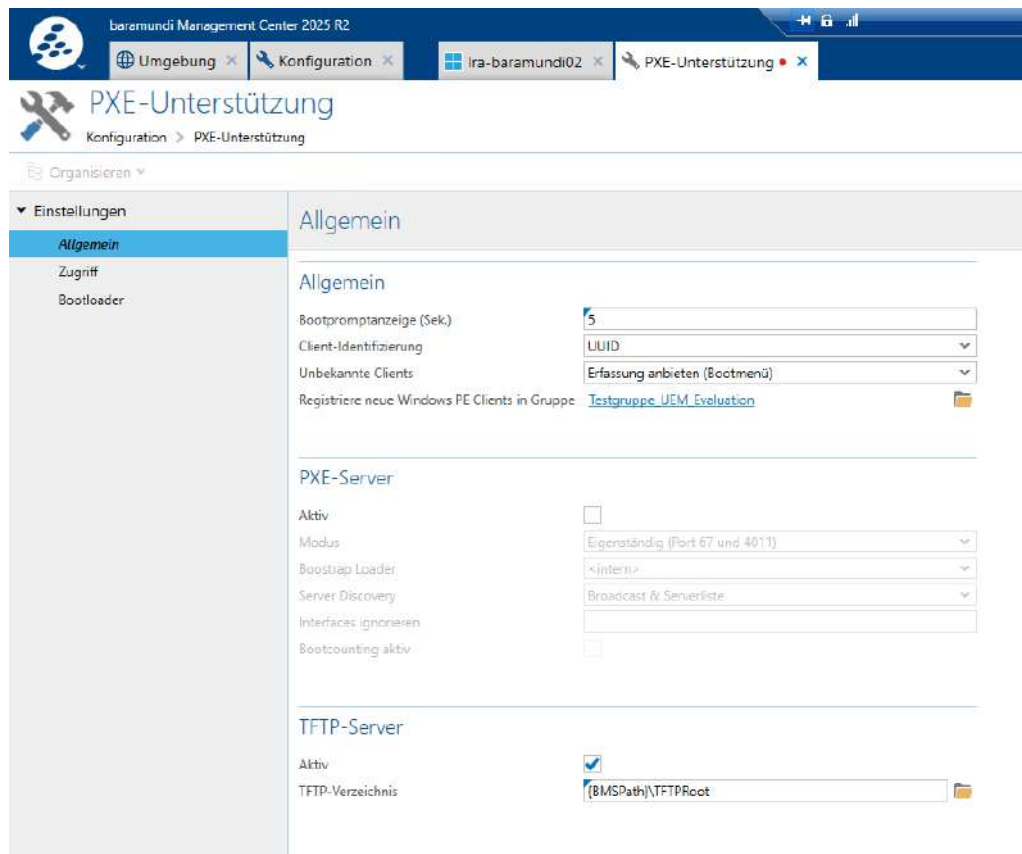


Abbildung 28: Konfiguration des PXE- und TFTP-Dienstes im baramundi Management Center

3.7 Vorbereitung des Windows-11-Deployments

3.7.1 Einbindung des Windows-11-Installationsmediums

Im nächsten Schritt wurde das Windows-11-Installationsmedium nativ in baramundi eingebunden. Die ISO-Datei diente als zentrale Quelle für das Betriebssystem-Deployment; die enthaltene install.wim wurde dabei als Betriebssystemabbild für die spätere unbeaufsichtigte Installation verwendet. Zusätzlich wurden Installationsparameter wie Edition, Zielpartition, Unattended-Datei und die Behandlung des Computerkontos festgelegt.

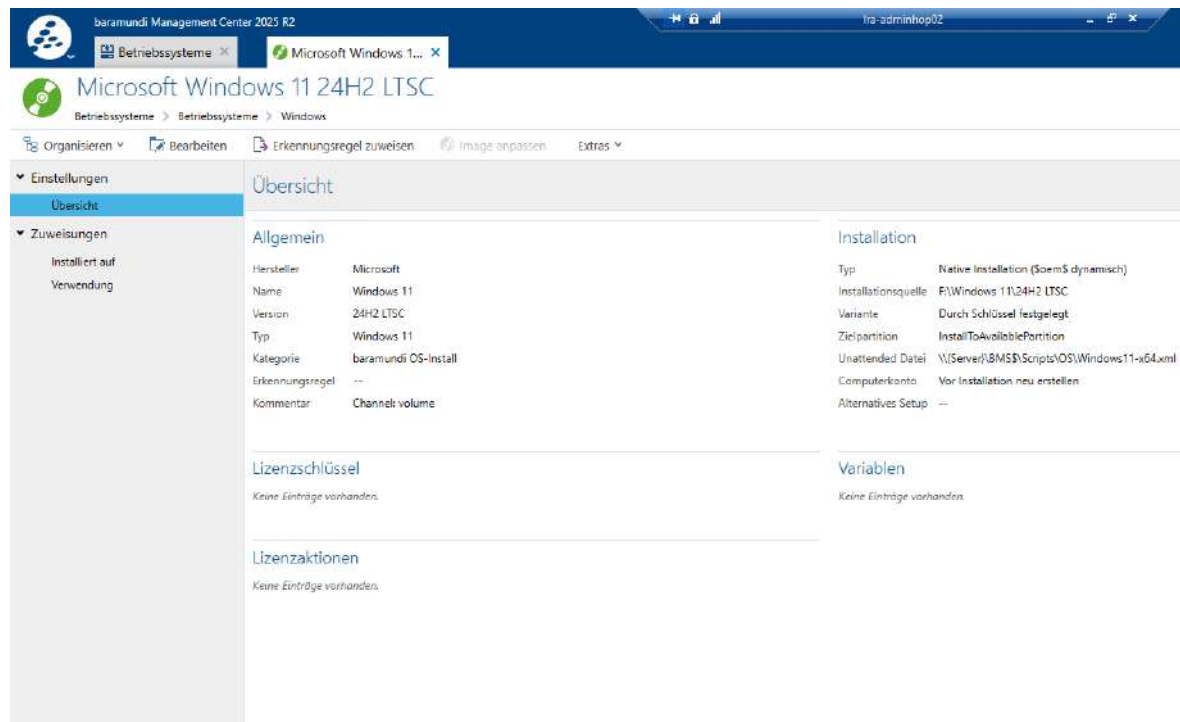


Abbildung 29: Native Einbindung des Windows-Installationsmediums in baramundi

3.8 Erstellung der WinPE-Bootumgebung

Für den PXE-Start wurde eine eigene WinPE-Bootumgebung benötigt. Vor der Erstellung mussten das Microsoft Windows ADK^[20], das WinPE Add-on und die DISM-Komponenten vollständig bereitstehen, da diese Werkzeuge die technische Grundlage für das Erzeugen und Anpassen der Bootumgebung bilden.

Erst nach der Einbindung des Windows-Installationsmediums wurde mit dem Boot Media Wizard^[6] die PXE-fähige WinPE-Umgebung erstellt. Dabei wurden Architektur, UEFI-Bootmodus, Windows-Kit-Pfad, Quell-WIM und Zielverzeichnis im TFTPRoot festgelegt. Der Wizard erzeugt daraus die boot.wim, die anschließend auf die PXE-/TFTP- beziehungsweise Deployment-Infrastruktur verteilt und später vom Client über PXE/TFTP geladen wird.

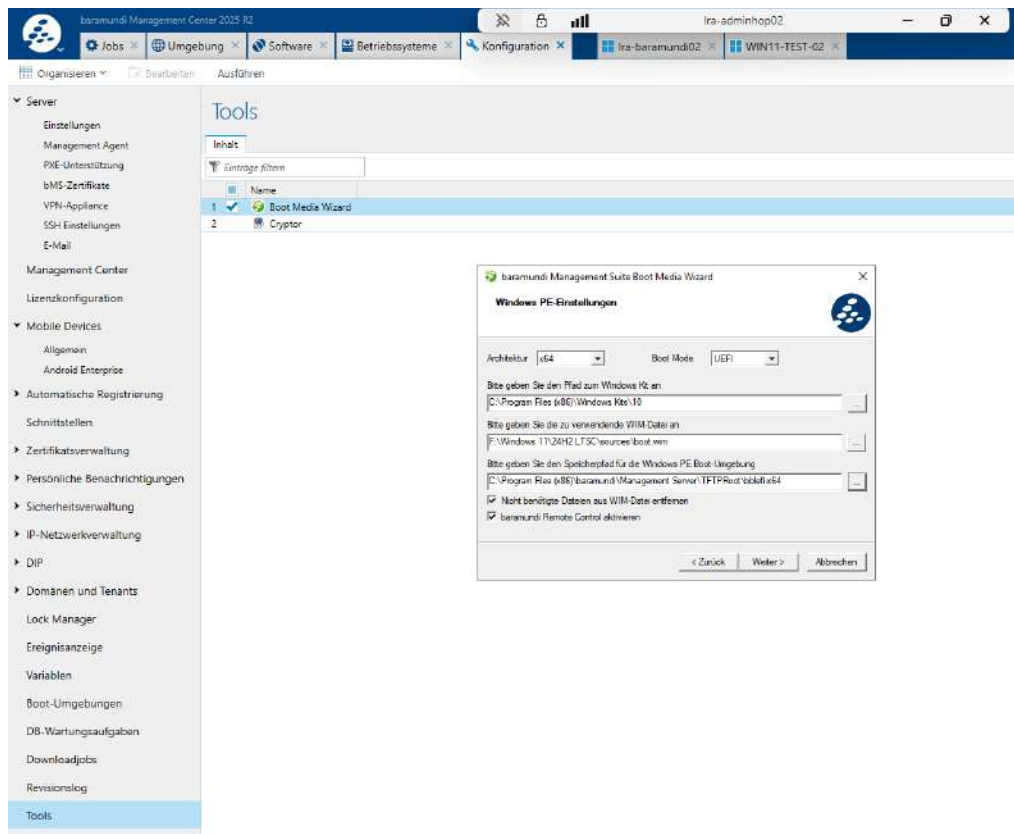


Abbildung 30: Boot Media Wizard - Erstellung der WinPE-Bootumgebung

Die anschließende Prüfung der Dateiablage bestätigte, dass die generierte boot.wim im TFTP-Verzeichnis bereitgestellt und damit als Boot-Image für die PXE-/TFTP-Infrastruktur verfügbar war. Die Datei stellt nicht das eigentliche Windows-Installationsabbild dar, sondern die WinPE-Startumgebung für den Netzwerkboot. Die Größe von rund 427.986 KB ist für eine WinPE-boot.wim mit Baramundi-Komponenten sowie Netzwerk- und Storage-Treibern plausibel.

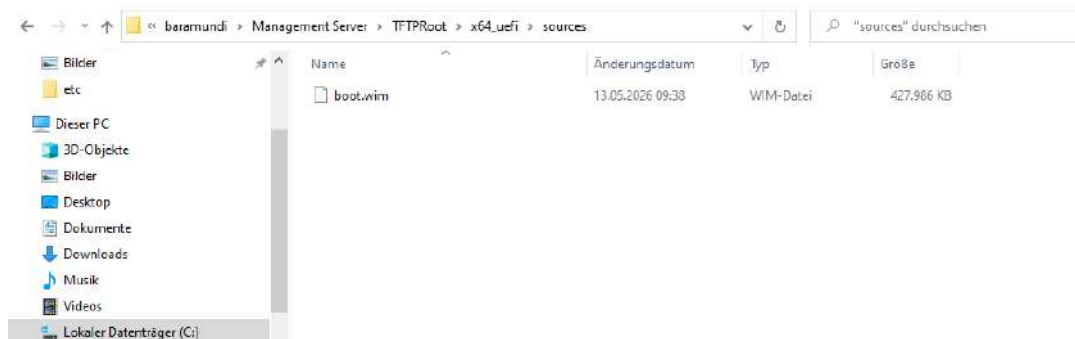


Abbildung 31: Dateisystem - Validierung der generierten Boot-Datei boot.wim im Verzeichnis TFTPRoot

Nach der Einbindung des Installationsmediums, der Erstellung der WinPE-Bootumgebung und der Bereitstellung des Boot-Images wurde ein eigener Installationsjob für die Client-Betriebssysteme angelegt.

Der Job fasst die für das Windows-11-Deployment erforderlichen Schritte zusammen und dient als steuerbare Einheit für die spätere Zuweisung an die Testgruppe beziehungsweise an Referenzclients.

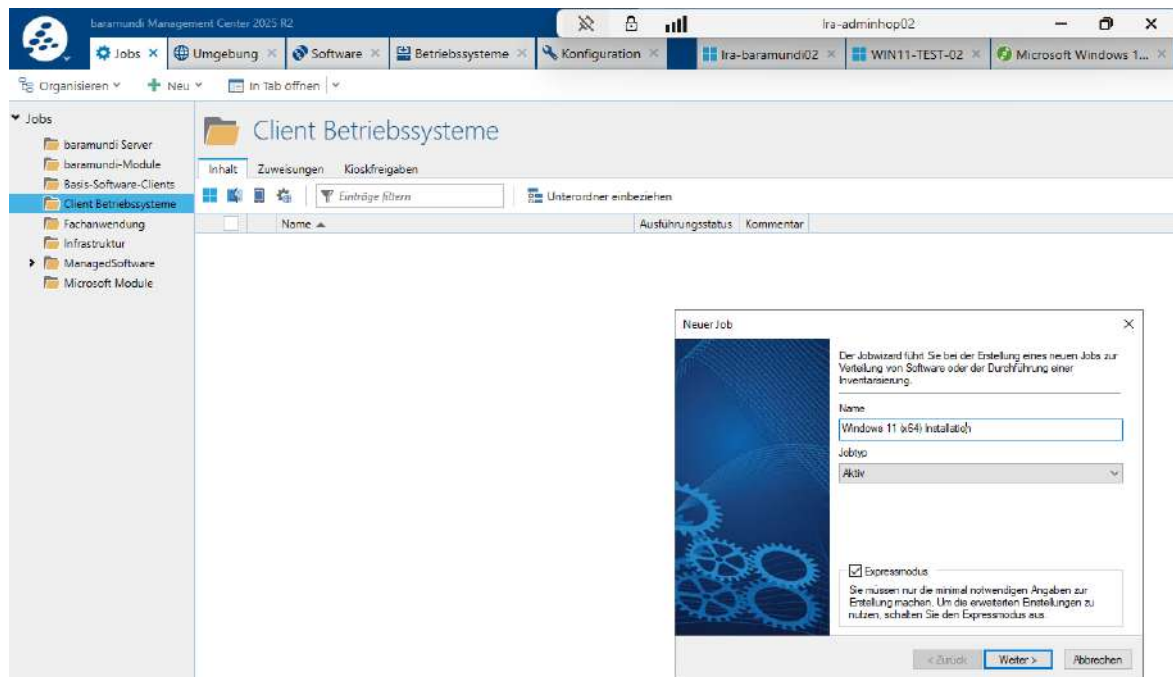


Abbildung 32: Erstellung des Windows-11-Installationsjobs für Client-Betriebssysteme

3.9 Erstellung eines Basis-Software-Bündels

3.9.1 Basis-Client-Software

Im Rahmen der Client-Basisbereitstellung wurde ein eigener baramundi-Job für die Installation der Standardsoftware erstellt. Dafür wurden die für den Testbetrieb benötigten Anwendungen und *Abhängigkeiten* zu einem gemeinsamen Installationsschritt zusammengefasst. Die Softwarepakete wurden über baramundi Managed Software bezogen, wodurch sie zentral bereitgestellt, aktualisiert und mit der passenden Installationslogik verwendet werden konnten.

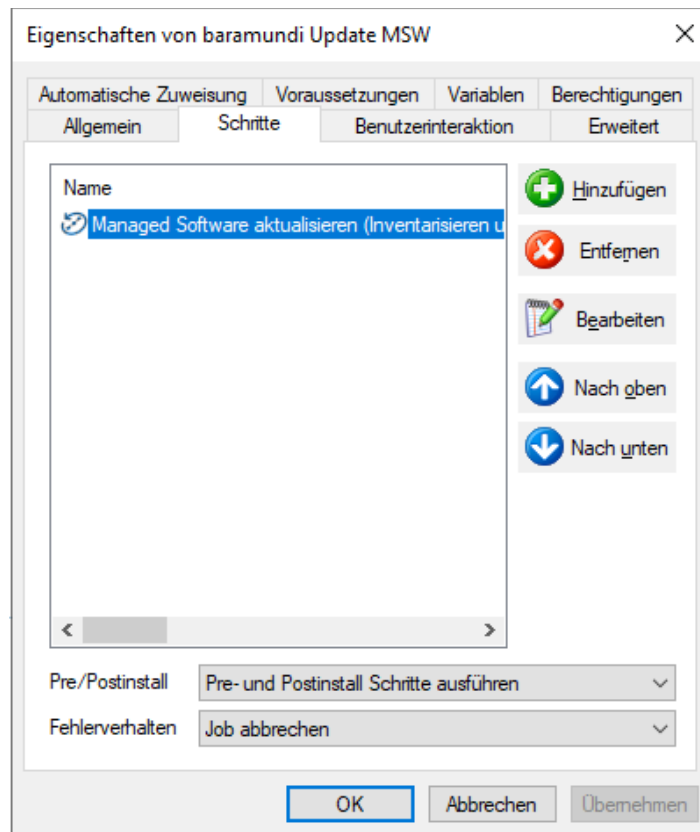


Abbildung 33: Erstellung eines Managed-Software-Jobs mit definiertem Installationsschritt

Der erstellte Job Basis_Software_Clients_Job enthält unter anderem Microsoft Visual C++ Redistributable, Microsoft Edge WebView2 Runtime, Citrix Workspace App, Cisco Webex, Adobe Reader DC und 7-Zip. Diese Komponenten wurden bewusst nicht in das Windows-Image integriert, sondern als nachgelagerter Softwarejob bereitgestellt.

Das Windows-11-Image und das Basis-Softwarepaket wurden modular getrennt aufgebaut. Das Betriebssystemimage bleibt dadurch schlank und wartungsarm, während Anwendungen unabhängig davon aktualisiert, ausgetauscht oder erweitert werden können. Softwareänderungen erfordern damit keine erneute Image-Erstellung, sondern können über baramundi-Jobs gesteuert werden.

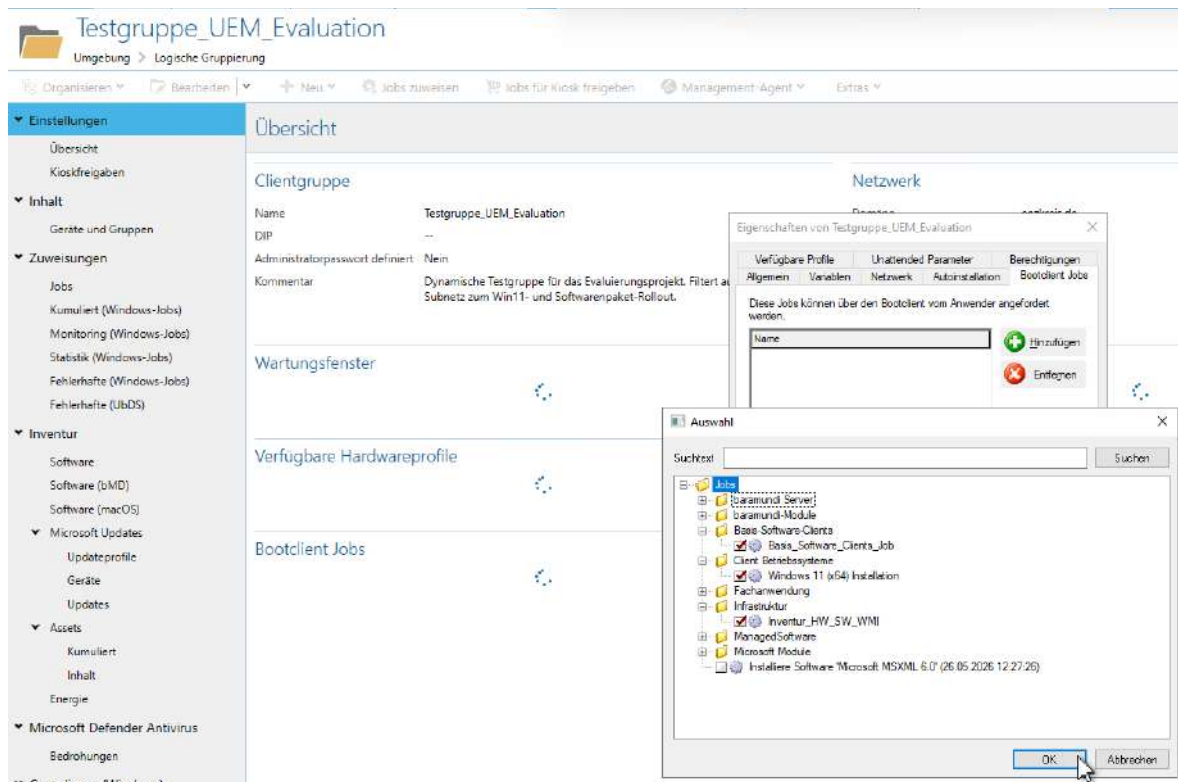


Abbildung 34: Zuweisung der erstellten Jobs zur Testgruppe_UEM_Evaluation

Nachdem der OS-Installationsjob^[11], das Basissoftware-Bündel und der Inventur-Job erstellt wurden, konnten diese der Testgruppe zugewiesen werden. Die Gruppenzuweisung bildete die Grundlage für den späteren Rollout auf die vorgesehenen Referenzclients.

3.10 Treiberintegration

Bei der Treibereinbindung wurde zwischen zwei Einsatzbereichen unterschieden. Für die WinPE-Bootumgebung werden insbesondere Netzwerk- und Storage-Treiber benötigt, damit der Client nach dem PXE-Start die Hardware initialisieren und den baramundi-Server beziehungsweise den DIP erreichen kann. Werden diese Treiber direkt im Boot Media Wizard in die *boot.wim* integriert, ist hierfür kein separates Treiberprofil erforderlich.

Davon getrennt sind die Treiber für das später installierte Windows-System zu betrachten. Für diese Postinstallation wurden modellbezogene Treiberprofile vorbereitet. Sie ermöglichen es, die passenden Treiber einem bestimmten Hardwaremodell beziehungsweise einem *OS-Installationsjob* zuzuordnen und während des Deployments kontrolliert bereitzustellen.

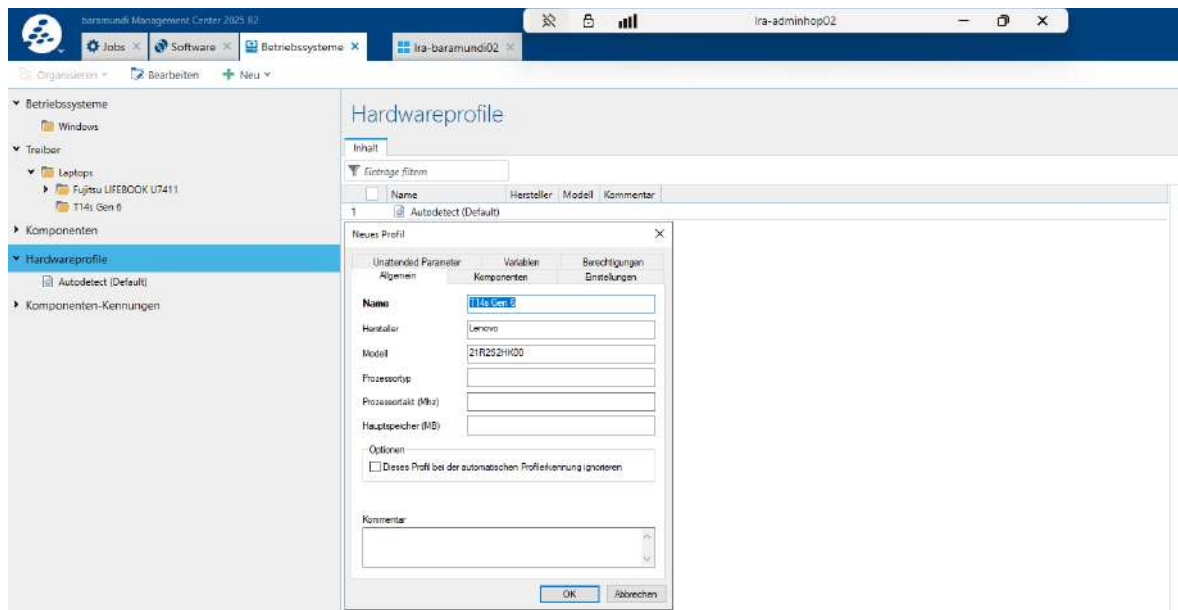


Abbildung 35: Erstellung eines Hardwareprofils für das Lenovo ThinkPad T14s Gen 6

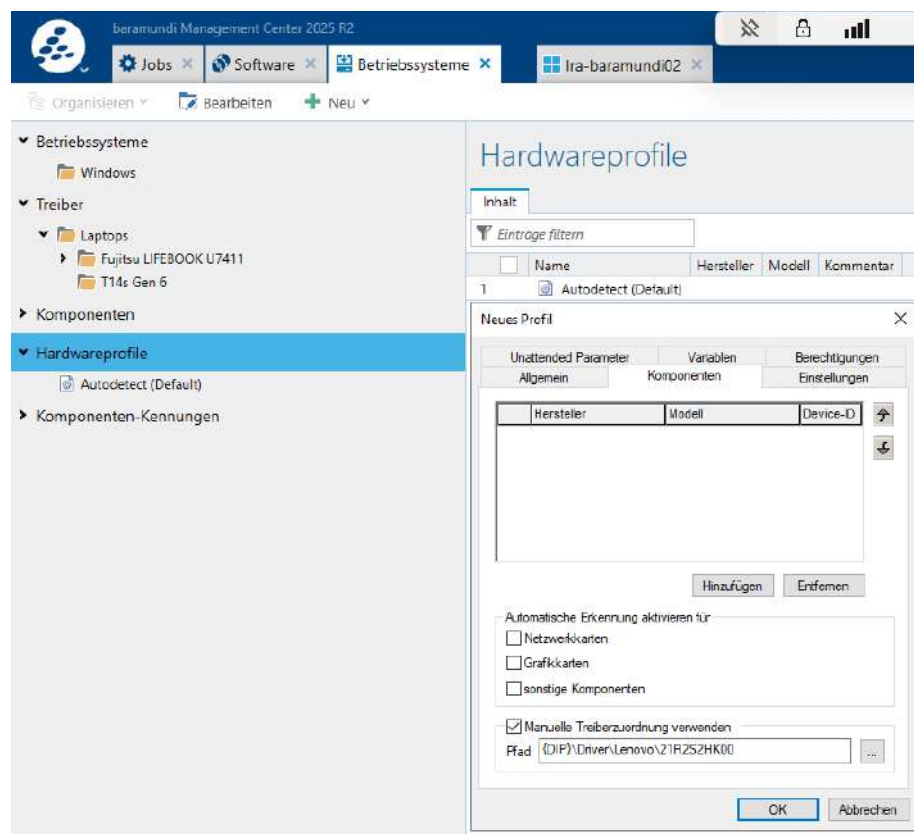


Abbildung 36: Manuelle Zuordnung des Treiberpfads zum Lenovo-Hardwareprofil

	Hersteller	Name	Version	Kategorie	Typ	Einträge
1	AlcorLink	AlcorLink USB Smart Card Reader (AKCCID.INF)	1.10.7.1001 (2F80000)	baramundi-OS-Install	Sonstige Komponente	112 Einträge
2	Dolby	Dolby Device Extension (dax3_ext_rtk.inf)	10.716.800.5 (2F80000)	baramundi-OS-Install	Sonstige Komponente	
3	Dolby	Dolby Device Extension (dax3_ext_rtk.inf)	8.1108.1220.0 (2F80000)	baramundi-OS-Install	Sonstige Komponente	
4	ELAN	TrackPoint Device (EPD.inf)	31.2151.2 (2F80000)	baramundi-OS-Install	Sonstige Komponente	
5	Goodix	Goodix Fingerprint Device Extension (goodixExtInf_SecbioOn.inf)	3.4.51.2020 (2F80000)	baramundi-OS-Install	Sonstige Komponente	
6	Goodix	Goodix MDC Fingerprint (goodixExtInf_UNP.inf)	3.4.6.110 (2F80000)	baramundi-OS-Install	Sonstige Komponente	
7	Goodix	Goodix MDC Fingerprint (GoodixMdcLib.inf)	3.4.51.2020 (2F80000)	baramundi-OS-Install	Sonstige Komponente	
8	INTEL	DmaSecurity AllowedBus Extension (AlderLakeCH-DmaSecExtension.inf)	10.1.45.10 (2FC0000)	baramundi-OS-Install	Sonstige Komponente	
9	INTEL	DmaSecurity AllowedBus Extension (AlderLakeCH-DmaSecExtension.inf)	10.1.50.8 (2FC0000)	baramundi-OS-Install	Sonstige Komponente	
10	INTEL	DmaSecurity AllowedBus Extension (AlderLakeCH-PDmaSecExtension.inf)	10.1.36.7 (2FC0000)	baramundi-OS-Install	Sonstige Komponente	
11	INTEL	Intel(R) LPC Controller/eSPI Controller - 7700 (ArrowLakeSystem.inf)	10.1.37.6 (2FC0000)	baramundi-OS-Install	Sonstige Komponente	
12	Intel	Intel(R) Active Management Technology - SOL (IntelAMT.inf)	2516.8.0.0 (2FC0000)	baramundi-OS-Install	Sonstige Komponente	
13	Intel	Intel(R) Context Sensing Technology Service Extension (icst_extension.inf)	2.1.1.0500.0007 (2F80000)	baramundi-OS-Install	Sonstige Komponente	
14	Intel	Intel(R) Dynamic Tuning Technology Extension (dtt_ext.inf)	9.0.11910.61833 (2F80000)	baramundi-OS-Install	Sonstige Komponente	
15	INTEL	Intel(R) Host Bridge/DRAM Registers - 4601 (AlderLakeSystem.inf)	10.1.45.10 (2FC0000)	baramundi-OS-Install	Sonstige Komponente	
16	Intel	Intel(R) Innovation Platform Framework Manager (ipf_acpi.inf)	2.2.10205.3620 (2F80000)	baramundi-OS-Install	Sonstige Komponente	
17	Intel	Intel(R) Innovation Platform Framework Processor Participant (ipf_cpu.inf)	2.2.10205.3620 (2F80000)	baramundi-OS-Install	Sonstige Komponente	
18	Intel	Intel(R) Innovation Platform Framework SMBUS Device (ipf_smbus.inf)	2.2.10205.3620 (2F80000)	baramundi-OS-Install	Sonstige Komponente	
19	Intel	Intel(R) Integrated Sensor Solution (ihs_ced.inf)	5.8.52.0 (2F80000)	baramundi-OS-Install	Sonstige Komponente	
20	INTEL	Intel(R) LPC Controller - 5180 (AlderLakeCH-System.inf)	10.1.36.7 (2FC0000)	baramundi-OS-Install	Sonstige Komponente	
21	INTEL	Intel(R) LPC Controller/eSPI Controller - 7A00 (RaptorLakeCH-System.inf)	10.1.40.5 (2FC0000)	baramundi-OS-Install	Sonstige Komponente	
22	INTEL	Intel(R) LPC Controller/eSPI Controller - 7A80 (AlderLakeCH-System.inf)	10.1.37.5 (2FC0000)	baramundi-OS-Install	Sonstige Komponente	
23	INTEL	Intel(R) LPSS SPI #2 - 51FB (AlderLakeCH-SystemLSS.inf)	10.1.36.7 (2FC0000)	baramundi-OS-Install	Sonstige Komponente	
24	INTEL	Intel(R) LPSS UART #0 - 7AAB (AlderLakeCH-SystemLSS.inf)	10.1.37.5 (2FC0000)	baramundi-OS-Install	Sonstige Komponente	
25	Intel	Intel(R) Management Engine Interface #1 (Ihccnt)	2346.6.9.0 (2FC0000)	baramundi-OS-Install	Sonstige Komponente	
26	INTEL	Intel(R) North Peak - 5146 (AlderLakeCH-SystemNorthpeak.inf)	10.1.36.7 (2FC0000)	baramundi-OS-Install	Sonstige Komponente	
27	INTEL	Intel(R) North Peak - 7AAB (AlderLakeCH-SystemNorthpeak.inf)	10.1.37.5 (2FC0000)	baramundi-OS-Install	Sonstige Komponente	
28	INTEL	Intel(R) North Peak - 7E24 (MeteorLakeSystemNorthpeak.inf)	10.1.47.12 (2FC0000)	baramundi-OS-Install	Sonstige Komponente	
29	INTEL	Intel(R) Northpeak - 466F (AlderLakeCH-SystemNorthpeak.inf)	10.1.45.10 (2FC0000)	baramundi-OS-Install	Sonstige Komponente	
30	INTEL	Intel(R) PCIe RC 010 G5 - A70D (RaptorLakeSystem.inf)	10.1.49.12 (2FC0000)	baramundi-OS-Install	Sonstige Komponente	
31	INTEL	Intel(R) SMBus - A822 (LunarLakeSystem.inf)	10.1.48.18 (2FC0000)	baramundi-OS-Install	Sonstige Komponente	
32	INTEL	Intel(R) SPI - 7E23 (MeteorLakeSystem.inf)	10.1.47.12 (2FC0000)	baramundi-OS-Install	Sonstige Komponente	
33	INTEL	North Peak - 5466 (AlderLakeCH-SystemNorthpeak.inf)	10.1.50.8 (2FC0000)	baramundi-OS-Install	Sonstige Komponente	
34	Intel	PCI Express Downstream Switch Port (mercent.inf)	1.1.1.1 (2F80000)	baramundi-OS-Install	Sonstige Komponente	
35	INTEL	SMBus - 54A3 (AlderLakeCH-System.inf)	10.1.50.8 (2FC0000)	baramundi-OS-Install	Sonstige Komponente	
36	INTEL	SMBus - 7F23 (MeteorLakeCH-System.inf)	10.1.51.10 (2FC0000)	baramundi-OS-Install	Sonstige Komponente	
37	Intel Corporation	H-D Audio Driver for Display Audio (HSDaDac.inf)	32.0.101.8332 (2F80000)	baramundi-OS-Install	Sonstige Komponente	
38	Intel Corporation	Intel(R) AI Boost (ipbu_extension.inf)	32.0.100.4512 (2F80000)	baramundi-OS-Install	Sonstige Komponente	

Abbildung 37: Importierte Windows-11-Treiber innerhalb des Lenovo-Hardwareprofils

Der typische Ablauf für die Postinstallation besteht damit aus Treiberimport, Erstellung eines Hardwarebeziehungsweise Treiberprofils und Zuordnung zum passenden Hardwaremodell oder OS-Installationsjob. Dadurch wird verhindert, dass Treiber ungezielt in das Betriebssystemabbild integriert werden müssen.

4. Testrollout und Fehlerbehebung

4.1 Testrollout und Fehleranalyse

Für die Pilotierung wurde die vorbereitete Testgruppe_UEM_Evaluation genutzt. Dieser Gruppe waren die erstellten Jobs für Betriebssysteminstallation, Basissoftwareverteilung und Inventarisierung zugeordnet. Dadurch konnte der Referenzclient im abgegrenzten Testumfang betankt und anschließend fachlich ausgewertet werden. Die Pilotierung bildet damit den direkten Übergang zum Testrollout und zur Fehleranalyse.

Im Testrollout wurde die vorbereitete baramundi-Bereitstellung mit einem Referenzclient geprüft. Zunächst trat ein Fehlverhalten auf, bei dem sich beim Client nicht der baramundi-Server, sondern weiterhin der bestehende *Empirum*-Server meldete. Da dieses Verhalten trotz geprüfter Konfiguration und ausdrücklichem Ausschluss des *Empirum*-Servers aus dem VLAN 33 auftrat, konnte die Ursache zunächst nicht eindeutig im baramundi-System eingegrenzt werden.

Die Fehlersuche erfolgte gemeinsam mit den zuständigen IT-Fachkräften im Netzwerksegment. Dabei wurden unter anderem DHCP-/PXE-Zuweisungen, Firewallregeln und die zugehörige Fachdokumentation geprüft. Da der *Empirum*-Server nicht ohne Abstimmung und ohne Risiko für den Produktivbetrieb aus dem laufenden Betrieb genommen werden konnte, musste für den Gegencheck ein Wartungsfenster von vier Arbeitstagen abgewartet werden. Nach dem kurzzeitigen Entfernen des *Empirum*-Servers aus dem Netz traten keine Interferenzen durch *Empirum* mehr auf.

Anschließend erreichte der Client zwar die PXE-Phase, der weitere Startvorgang lief jedoch nicht erfolgreich in die baramundi-Startumgebung. Fachlich war damit nicht die DHCP-/PXE-Grundkonfiguration die entscheidende Fehlerstelle, sondern die nachgelagerte WinPE-Initialisierung. Die verwendete WinPE-Bootumgebung enthielt keinen kompatiblen Netzwerktreiber für die Hardware des Testclients. Dadurch konnte der Netzwerkadapter nach dem Start von WinPE nicht korrekt initialisiert werden, obwohl der PXE-Prozess zuvor grundsätzlich angestoßen wurde.

Nach der Neuerstellung des WinPE-Boot-Images und der Integration der passenden Netzwerktreiber startete der PXE-Boot erfolgreich über baramundi. Die Vorgehensweise entspricht der in Abschnitt 3.8 beschriebenen Erstellung der WinPE-Bootumgebung.

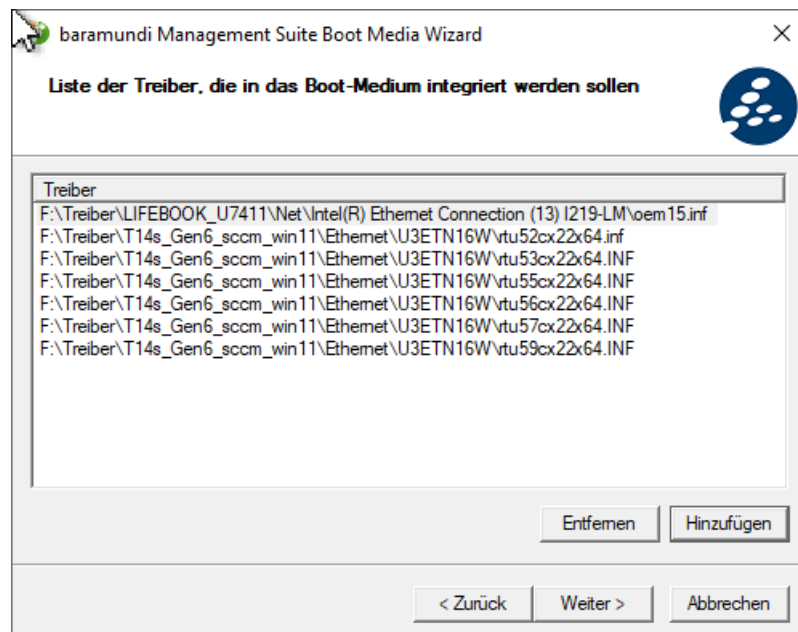


Abbildung 38: Integration der Netzwerktreiber in die WinPE-Bootumgebung über den Boot Media Wizard

Die Installation lief anschließend weiter, brach jedoch wiederholt während der Windows-Setup-Phase ab.



Abbildung 39: Windows-Setup - Abbruch der unbeaufsichtigten Installation auf dem Testclient

Zur Eingrenzung wurden anschließend die Setup-Protokolle der Windows-Installation ausgewertet:

```
dir C:\Windows\Panther
```

```
type C:\Windows\Panther\setuperr.log
```

```
type C:\Windows\Panther\setupact.log
```

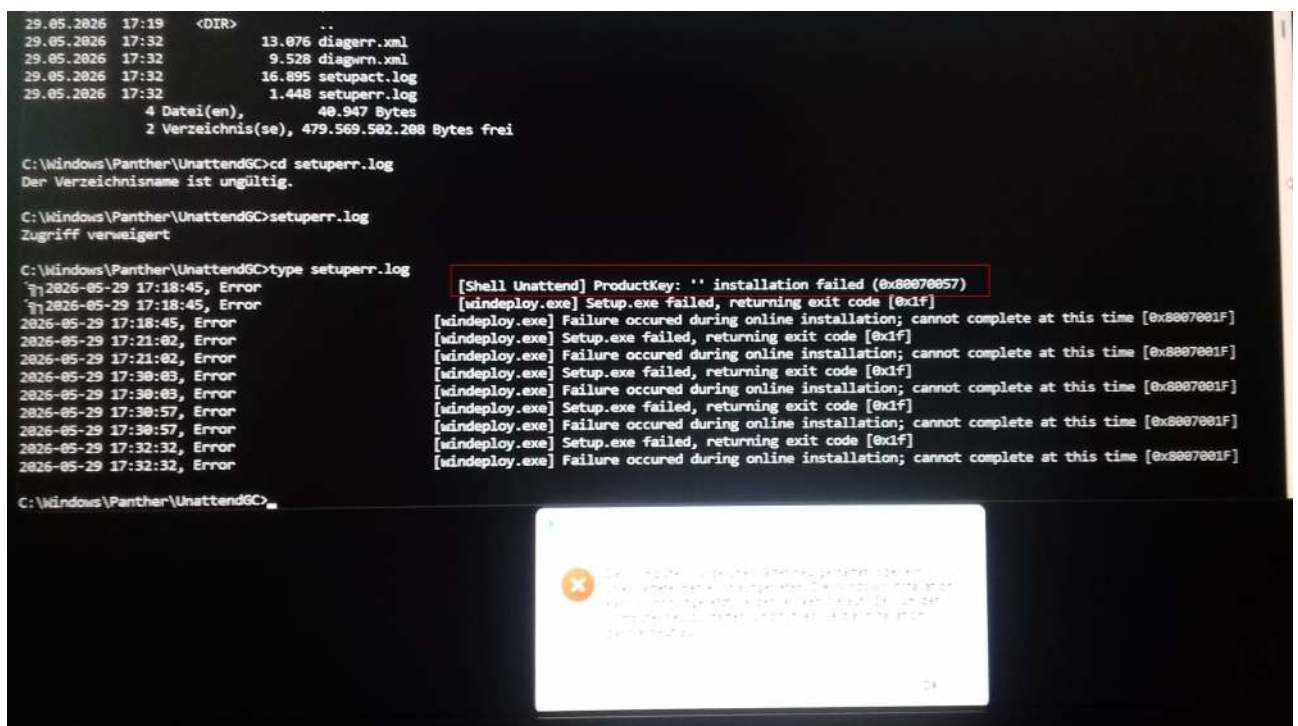


Abbildung 40: Auswertung der Setup-Protokolle mit Hinweis auf den fehlenden ProductKey

Die Protokollauswertung zeigte den Fehlerhinweis „ProductKey: " installation failed (0x80070057)“. Ursache war ein fehlender Produktschlüssel in der unbeaufsichtigten Windows-Installation. In baramundi wurde daraufhin ein generischer Microsoft-Installationsschlüssel für die verwendete Windows-Edition hinterlegt. Dieser Schlüssel dient nicht der Aktivierung, sondern ermöglicht die editionsbezogene Installation und den sauberen Durchlauf des Setups.

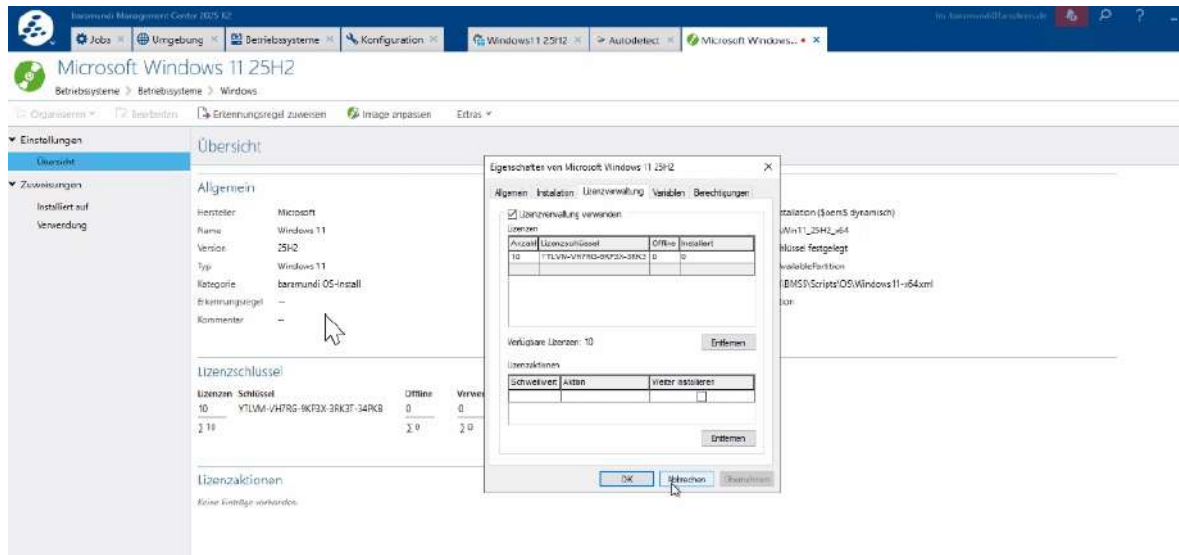


Abbildung 41: Hinterlegung des generischen Installationsschlüssels im Windows-Installationsobjekt

Nach dieser Korrektur konnten die vier im verfügbaren Zeitfenster getesteten Clients erfolgreich betankt werden. Nach dem Betriebssystem-Rollout wurde der baramundi Management Agent auf dem Client bereitgestellt. Dieser Agent ist die Voraussetzung dafür, dass nachgelagerte baramundi-Jobs wie Softwareinstallationen, Inventarisierung und Statusrückmeldungen auf dem Zielsystem ausgeführt und an die Management Suite zurückgemeldet werden können. Die Zuweisung über die Gruppe Test_UEM_Evaluation funktionierte ohne weitere funktionale Störungen. Der Betriebssystem-Rollout, die Verteilung der Basis-Client-Software sowie die Inventarisierungsjobs liefen im anschließenden Testlauf nachvollziehbar durch.

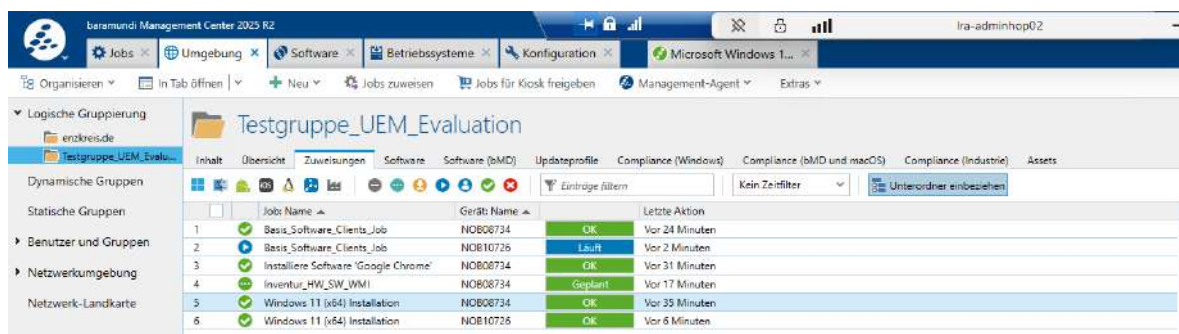


Abbildung 42: Erfolgreich abgeschlossene Rollout (inklusive Basissoftware-Jobs+ Inventur-Job) im Testlauf

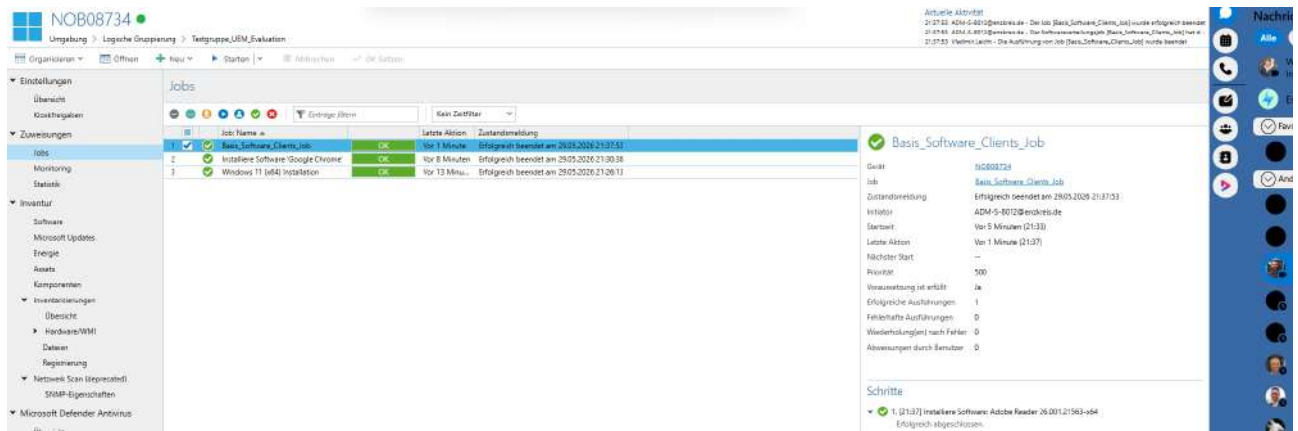
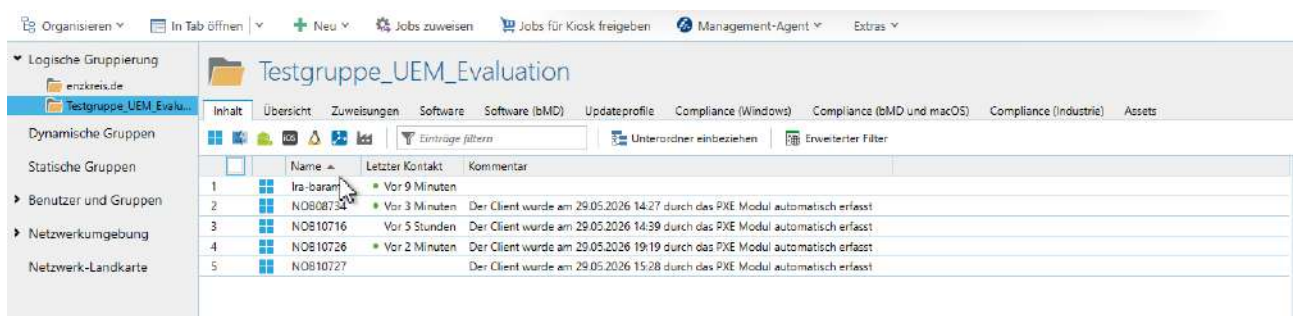


Abbildung 43: Erfolgreich abgeschlossene Rollout Nr. 2 (inklusive Basissoftware-Jobs+ Inventur-Job) im Testlauf



Abbild 44: Erfolgreich Clienterfassung in der Testgruppe

5. Projektabschluss

5.1 Soll-Ist-Vergleich

Der Vergleich zwischen den geplanten Zielsetzungen und den tatsächlich erreichten Ergebnissen zeigt, dass der Projektrahmen im Wesentlichen vollständig umgesetzt wurde. Die Bereitstellung des On-Premises-bMS-

Servers mit Anbindung an die MSSQL-Datenbank wurde erfolgreich realisiert. Die Kommunikation zwischen Server und Client erfolgt dabei verschlüsselt über TLS auf Port 443, wodurch die Verbindung den üblichen Sicherheitsanforderungen entspricht.

Auch die fachlichen Kernziele wurden erreicht. Das automatisierte Windows-11-*PXE*-Deployment konnte so eingerichtet werden, dass die Installation der Zielsysteme reproduzierbar angestoßen werden kann. Die Bereitstellung der definierten Standardanwendungen über *bDeploy* verlief grundsätzlich wie geplant. Die im Projektantrag vorgesehene Anzahl von fünf Referenzclients wurde aus Zeitgründen nicht vollständig erreicht; vollständig betankt wurden vier Clients. Nach Behebung der im Testrollout beschriebenen Fehler liefen die tatsächlich durchgeführten vier Betankungen ohne weitere funktionale Störungen durch. Bezogen auf diese vier abgeschlossenen Testläufe ergibt sich eine Erfolgsquote von 100 %, was durch die im Abschnitt 4.1 eingebundenen Ergebnis-Screenshots belegt wird. Die funktionale Zielsetzung des Pilotrollouts wurde dadurch nicht aufgehoben, da Betriebssysteminstallation, Basissoftwareverteilung und Inventarisierung erfolgreich nachgewiesen wurden.

5.2 Zeitmanagement

Die Projektarbeit wurde in getrennten Zeitfenstern durchgeführt. Die Netto-Projektzeit blieb dabei auf 40 Stunden begrenzt. Wartezeiten, Freigaben und Tätigkeiten anderer Fachbereiche wurden nicht als Projektzeit des Prüflings gewertet; berücksichtigt wurden ausschließlich die unmittelbar für Planung, Umsetzung, Prüfung, Recherche und Dokumentation aufgewendeten Zeiten.

Da die Arbeiten im *PoC*-Subnetz den laufenden Betrieb nicht beeinträchtigen durften, hatten produktive Betriebsprozesse Vorrang. Phasen ohne unmittelbaren technischen Fortschritt wurden deshalb für Recherche, Informationsbeschaffung, Prüfung technischer *Abhängigkeiten* und die projektbegleitende Vorbereitung der Dokumentation genutzt, sofern sich die Aufgaben nicht mit anderen Tätigkeit und Terminen, die der Prüfling stellenweise hatte, überschneit.

Zeitfenster	Tätigkeitsschwerpunkt	Nettozeit
08.05.	Grundlagen für die Serverbereitstellung in vSphere und Vorbereitung der technischen Ausgangsbasis	1,6 h
11.05.–12.05.	Installation und Grundkonfiguration von Windows Server, baramundi Management Suite, Datenbankanbindung und DIP	12,0 h
13.05.–19.05.	Eingrenzung der Empirum-	7,4 h

	Interferenz, Recherche, Informationsbeschaffung und Aufbau des Dokumentationsgrundgerüsts während unterbrochener Projektphasen; davon 2,4 h Dokumentationsvorbereitung und 5,0 h Puffer-/Rechercheanteil	
20.05.	Festlegung des weiteren Vorgehens und Abstimmung des erforderlichen Wartungsfensters als Puffer-/Abstimmungsanteil	1,0 h
21.05.–28.05.	Pufferphase für Recherche, Prüfung technischer Abhängigkeiten und Abstimmung bei weiterhin eingeschränkter Umsetzungsmöglichkeit	2,0 h
29.05.	Wartungsfenster mit Abschaltung des Empirum-Servers, finaler Testrollout, Treiberanpassung, ProductKey-Korrektur und Validierung	8,0 h
31.05.	Fertigstellung, formaler Abgleich und Abschlusskorrektur der Projektdokumentation	8,0 h
Gesamt		40,0 h

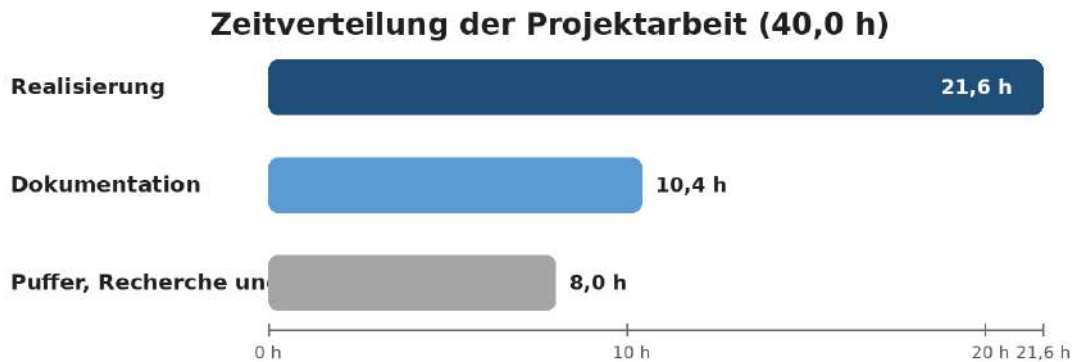


Abbildung 45: Zeitverteilung der Projektarbeit nach Nettoaufwand

Die Realisierung umfasste damit 21,6 Stunden. Die Dokumentation setzte sich aus 2,4 Stunden projektbegleitender Dokumentationsvorbereitung und 8,0 Stunden Abschlusskorrektur zusammen und betrug insgesamt 10,4 Stunden. Die verbleibenden Puffer-, Recherche- und Abstimmungsanteile beliefen sich auf 8,0 Stunden. Der verlängerte Bearbeitungszeitraum führte dadurch nicht zu einer Überschreitung der zulässigen Projektzeit.

5.3 Fazit und Ausblick

5.3.1 Fazit

Die Migration der Endpoint-Management-Infrastruktur von *Matrix42 Empirum* auf die *baramundi Management Suite* wurde innerhalb des vorgesehenen Zeitrahmens erfolgreich abgeschlossen. Die zentralen technischen Anforderungen konnten ohne grundlegende Abweichungen vom Projektplan umgesetzt werden. Dazu zählen insbesondere das stabilisierte Windows-11-Deployment über UEFI-PXE, die Bereitstellung der Standardsoftware über *bDeploy* und die Einbindung der grundlegenden Verwaltungs- und Patchmanagement-Funktionen.

Nach der Fehlerbehebung verliefen die relevanten Deployments im Testbetrieb erfolgreich. Das bedeutet nicht, dass jeder Einzelvorgang generell risikofrei wäre, innerhalb des definierten Projektumfangs war das Ergebnis jedoch stabil und nachvollziehbar.

5.3.2 Ausblick

Für die nächste Phase ist vorgesehen, den *baramundi Management Agent* schrittweise auf die übrige Client-Landschaft der Behörde auszuweiten. Darüber hinaus soll geprüft werden, ob das Modul für Mobile Device Management in einer späteren Projektstufe sinnvoll eingesetzt werden kann, um auch mobile Endgeräte zentral und einheitlich zu verwalten.

Die wirtschaftliche Betrachtung zeigt, dass sich die Umstellung durch reduzierte Bearbeitungszeiten und eine bessere Standardisierung positiv auf die laufenden Betriebskosten auswirken kann. Zusätzlich kann die Verantwortung für Paketierung und Softwarebereitstellung auf mehrere Beschäftigte verteilt werden. Dadurch sinkt die Abhängigkeit von einzelnen Spezialisten, während die Flexibilität der betrieblichen Abläufe steigt.

5.4 Lessons Learned / Reflexionen

Aus dem Projekt ergab sich neben den technischen Erkenntnissen auch ein besserer Einblick in die Arbeitsprozesse einer professionellen IT-Umgebung. Die Umsetzung zeigte, dass eine qualifizierte und regelmäßige Abstimmung zwischen den beteiligten Fachbereichen eine wesentliche Voraussetzung für die technische Lösungsfindung ist. Insbesondere bei Netzwerksegmentierung, DHCP-/PXE-Abhängigkeiten, Dienstknoten und Serverbereitstellung sollten Zuständigkeiten, Voraussetzungen und Änderungen möglichst früh geklärt werden.

Zudem wurde deutlich, dass Geduld und systematisches Vorgehen bei der Fehleranalyse entscheidend sind. Nicht jede Störung ließ sich unmittelbar einer einzelnen Anwendungskomponente, einem einzelnen Prozess oder einer einzelnen Konfiguration zuordnen. Erst durch das schrittweise Prüfen von Serverkonfiguration, Netzwerkpfaden, Berechtigungen und baramundi-Einstellungen konnten Ursachen eingegrenzt und nachvollziehbar behoben werden.

Die praktischen Kenntnisse und Fähigkeiten konnten insbesondere in folgenden Bereichen erweitert werden:

- **Virtualisierungsumgebungen**
 - Bereitstellung und Prüfung der Server-VM in vSphere
 - Zusammenhang zwischen VM-Konfiguration, Netzwerkanbindung und Betriebssysteminstallation
- **Windows-Server-Grundkonfiguration**
 - Konfiguration einer festen IP-Adresse
 - DNS-Anbindung des Servers
 - Vorbereitung des Servers für den Betrieb der baramundi Management Suite
- **Active Directory und Berechtigungen**

- Arbeit mit Administrationsgruppen
- Einrichtung von Dienstkonten und lokalen Berechtigungen
- Bedeutung eines nachvollziehbaren Rechtekonzepts für Installation, Betrieb und Fehleranalyse
- **Netzwerk- und Deployment-Abhängigkeiten**
 - Abstimmung von DHCP, PXE und TFTP
 - Berücksichtigung von VLAN- und DMZ-Segmentierung
 - Anbindung und Test von Clients
 - Erkenntnis, dass OS-Deployment nicht isoliert auf dem Management-Server betrachtet werden kann
- **baramundi-Architektur**
 - Zusammenspiel von Management Server, DIP/baraDIP, bMA, bDeploy, Patchmanagement und Inventarisierung
 - Nachvollziehbarkeit der Architektur durch praktische Einrichtung und Pilotbetrieb
- **Strukturierte Fehleranalyse**
 - Analyse des IPv4-/IPv6- bzw. vSphere-Netzwerkproblems
 - Erkenntnis, dass Fehlersymptome häufig nicht an der Stelle entstehen, an der sie sichtbar werden.

Für künftige Projekte ist daher eine noch frühere Abstimmung der technischen Rahmenbedingungen sowie eine konsequente Dokumentation von *Abhängigkeiten* sinnvoll.

Im Rahmen der dokumentarischen Aufarbeitung wurden die technischen Prozesse vertieft nachvollzogen, die durch die jeweiligen GUI-Aktionen ausgelöst werden. Die folgende Übersicht zeigt die komprimierten Schritte und Hintergrundprozesse dieses OS-Deployment-Lebenszyklus:



Schritt 1: Integration Windows-ISO in baramundi

Import der Windows-ISO: Einlesen des originalen ISO-Mediums [*Windows-ISO: Standard-Abbilddatei des Betriebssystems. Schlagwörter: Quellmedium, ISO-Abbild*].

Extraktion der install.wim: Entpacken des zentralen Datei-Containers [install.wim: Dateibasiertes Microsoft-Image-Format für Systemdateien. Schlagwörter: WIM^[22], Image-Datei].

Image-Analyse: Auslesen der enthaltenen Windows-Editionen zur Auswahl.

Schritt 2: WIM-Aufbereitung / Image-Basis

Editions-Auswahl: Selektion der Ziel-Version (z. B. Windows 11 Enterprise LTSC) [*Windows-Edition: Betriebssystem-Variante für den Unternehmenseinsatz. Schlagwörter: LTSC, Windows-Version*].

Vorbereitung: Bereitstellen für Offline-Anpassungen durch Mounten [Mounten: Einhängen eines virtuellen Abbilds zur Offline-Modifikation. Schlagwörter: DISM^[21], Offline-Mount].

bOCT^[10]-Übergabe: Bereitstellung der WIM-Basis im OS Customization Tool [OS Customization Tool: baramundi-Konfigurator zur Image-Anpassung. Schlagwörter: bOCT, Image-Vorbereitung].

Schritt 3: Image-Anpassung (Optional)

Treiber-Integration: Offline-Injektion hardwarenaher INF-Treiber [Hardware-Treiber: Systemnahe Treiber für Chipsatz, Netzwerk und Speicher. Schlagwörter: INF-Integration, DISM-Injektion].

Updates einbinden: Integration kumulativer Sicherheits-Updates [*Windows-Updates: Offline-Patch-Integration zur Verringerung des anschließenden Patch-Aufwands. Schlagwörter: Slipstreaming, Patch-Integration*].

Speichern: Unmounten und dauerhaftes Sichern der Änderungen (Commit) [Commit: Speichern der vorgenommenen Änderungen im WIM-Image. Schlagwörter: Save-Changes, Commit-Aktion].

Schritt 4: Bereitstellung im baramundi OS-Install-Modul

Image-Import: Einlesen des angepassten WIM-Images in das OS-Install-Modul [baramundi OS-Install: baramundi-Modul zur automatisierten Betriebssystem-Bereitstellung. Schlagwörter: OS-Deployment, bMS-Modul].

Job-Verknüpfung: Zuweisung des Images zu den entsprechenden Installations-Jobs [Deployment-Jobs: Nacheinander ausgeführte Installationsschritte auf dem Client. Schlagwörter: bMS-Job, Task-Kette].

Zielgruppen-Definition: Verknüpfung des Jobs mit den Ziel-Hardwareprofilen [*Zielgruppen / Profile: Bedingungen zur dynamischen Verteilung an Endgeräte. Schlagwörter: Client-Gruppe, Profil-Zuweisung*].

Schritt 5: Erstellung des Boot-Mediums

WinPE-Startumgebung: Erstellung eines minimalistischen Boot-Betriebssystems im RAM [*WinPE*: Minimal-Betriebssystem im Arbeitsspeicher zur Vorbereitung des Setups. Schlagwörter: Preinstallation, RAM-Disk].

PE-Treiber-Integration: Einbindung essenzieller Netzwerk- und Storage-Treiber [Boot-Treiber: NIC- und Controller-Treiber, die für den *WinPE*-Start zwingend sind. Schlagwörter: NIC-Treiber, Storage-Driver].

Wizard-Ausgabe: Erstellung des Startmediums (*PXE*- oder USB-Medium) mittels *Boot Media Wizard* [*Boot Media Wizard*: Assistent zur Erstellung startfähiger Medien. Schlagwörter: bBMW, ISO-Ausgabe].

Schritt 6: Client-Start und Installation

WinPE-Boot: Start des Clients in die *WinPE*-Umgebung via *PXE* oder USB [*WinPE*-Boot: Startphase des Clients in die *WinPE*-Laufzeitumgebung. Schlagwörter: Boot-Phase, PE-Start].

Image-Abruf: Aufbau der Netzwerkverbindung und Herunterladen des *WIM-Images* vom DIP.

Setup-Durchlauf: Extraktion des *WIM-Images* und vollautomatisierte Installation (Unattended Setup^[12]) [Unattended Setup: Unbeaufsichtigte Windows-Installation über eine Antwortdatei (autounattend.xml^[23]). Schlagwörter: autounattend.xml, Silent-Install].

Anhang A: Begriffserläuterungen und Quellenangaben

1 Matrix42 Empirum: Endpoint-Management- und Softwareverteilungslösung, die im Projekt als bisherige Clientverwaltungsumgebung abgelöst wurde. [Quelle: Matrix42-Dokumentation](#). [\[zurück zur Stelle im Dokument\]](#)

2 baramundi Management Suite (bMS): Unified-Endpoint-Management-Plattform zur zentralen Verwaltung von Inventarisierung, Softwareverteilung, Patchmanagement und Betriebssystembereitstellung. [Quelle: baramundi Produktseite](#). [\[zurück zur Stelle im Dokument\]](#)

3 Distributed Installation Point (DIP) / baraDIP: baramundi-Komponente zur Bereitstellung und Synchronisation von Installationsquellen, Softwarepaketen und Deployment-Daten. [Quelle: baramundi Systemanforderungen](#). [\[zurück zur Stelle im Dokument\]](#)

4 bConnect-Schnittstelle: HTTPS-basierte Schnittstelle der baramundi Management Suite zur Anbindung externer Systeme und Komponenten. [Quelle: baramundi Interfaces](#). [\[zurück zur Stelle im Dokument\]](#)

5 bDeploy: baramundi-Funktion zur Paketierung und automatisierten Bereitstellung von Software auf verwalteten Endgeräten. [Quelle: baramundi Management Suite](#). [\[zurück zur Stelle im Dokument\]](#)

6 Boot Media Wizard: Assistent zur Erstellung startfähiger WinPE-Medien für PXE- oder USB-basierte Betriebssystembereitstellung. [Quelle: baramundi Help Center](#). [\[zurück zur Stelle im Dokument\]](#)

7 PXE/TFTP: Verfahren für den Netzwerkstart eines Clients und die Übertragung der dafür benötigten Bootdateien. [Quelle: Microsoft PXE-Troubleshooting](#). [\[zurück zur Stelle im Dokument\]](#)

8 baramundi Management Agent (bMA): Clientkomponente der baramundi-Umgebung, über die Inventarisierung, Jobausführung und Softwarebereitstellung auf Endgeräten gesteuert werden. [Quelle: baramundi Management Suite](#). [\[zurück zur Stelle im Dokument\]](#)

9 bBT: In der baramundi-Umgebung genutzter Übertragungsmechanismus für Datenzugriffe zwischen Client, Dienstkonto und DIP-Struktur. [Quelle: baramundi Systemanforderungen](#). [\[zurück zur Stelle im Dokument\]](#)

10 boCT / OS Customization Tool: baramundi-Werkzeug zur Vorbereitung und Anpassung von Windows-Images im Rahmen der Betriebssystembereitstellung. [Quelle: baramundi Help Center](#). [\[zurück zur Stelle im Dokument\]](#)

11 baramundi OS-Install: baramundi-Modul zur automatisierten Betriebssysteminstallation und Verknüpfung von Images mit Installationsjobs. [Quelle: baramundi Management Suite](#). [\[zurück zur Stelle im Dokument\]](#)

12 Unattended Setup: Unbeaufsichtigte Windows-Installation, bei der Einstellungen über eine Antwortdatei automatisiert vorgegeben werden. [Quelle: Microsoft Unattended Setup Reference](#). [\[zurück zur Stelle im Dokument\]](#)

13 ASP.NET Core Runtime: Microsoft-Laufzeitumgebung zum Ausführen von ASP.NET-Core-Anwendungen und webbasierten Serverkomponenten. [Quelle: Microsoft .NET-Installation](#). [\[zurück zur Stelle im Dokument\]](#)

14 Software-Abhängigkeiten: Technische Voraussetzungen wie Laufzeitumgebungen, Bibliotheken oder Dienste, die für Installation und Betrieb einer Anwendung erforderlich sind. Quelle: Projektkontext. [\[zurück zur Stelle im Dokument\]](#)

15 ErrorLevel 3010: Rückgabecode-Auswertung in Skripten; der Code 3010 steht typischerweise für eine erfolgreiche Installation mit erforderlichem Neustart. [Quelle: Microsoft Windows Installer Error Messages](#). [\[zurück zur Stelle im Dokument\]](#)

16 MSW- und Patch-Quellen: baramundi-bezogene Quellen für Managed Software und Patchdaten, die über den DIP bereitgestellt beziehungsweise verarbeitet werden. [Quelle: baramundi Managed Software](#). [\[zurück zur Stelle im Dokument\]](#)

17 BDS-Dateien: baramundi-bezogene Daten- beziehungsweise Steuerdateien, die von den Serverdiensten gelesen und verarbeitet werden. Quelle: Projektkontext baramundi. [\[zurück zur Stelle im Dokument\]](#)

18 DIP\$: Administrative Windows-Freigabe des Distributed Installation Point für serverseitige Zugriffe auf Installations- und Paketdaten. [Quelle: baramundi Systemanforderungen](#). [\[zurück zur Stelle im Dokument\]](#)

19 bCenter-Verbindung: Verbindung der baramundi Management Console beziehungsweise Serverkomponenten zur bMS-Datenbank und Verwaltungsumgebung. [Quelle: baramundi Help Center](#). [\[zurück zur Stelle im Dokument\]](#)

20 Windows ADK: Microsoft-Werkzeugsammlung für Windows-Bereitstellung, Image-Anpassung und Erstellung von WinPE-Umgebungen. [Quelle: Microsoft Learn](#). [\[zurück zur Stelle im Dokument\]](#)

21 DISM: Deployment Image Servicing and Management; Microsoft-Werkzeug zum Bereitstellen, Prüfen und Ändern von Windows-Images. [Quelle: Microsoft Learn](#). [\[zurück zur Stelle im Dokument\]](#)

22 WIM-Image: Dateibasiertes Windows-Image-Format zur Ablage und Verteilung von Windows-Installations- oder Startabbildern. [Quelle: Microsoft DISM-Dokumentation](#). [\[zurück zur Stelle im Dokument\]](#)

23 autounattend.xml: Antwortdatei für die unbeaufsichtigte Windows-Installation, in der Setup-Optionen automatisiert vorgegeben werden. [Quelle: Microsoft Unattended Setup](#). [\[zurück zur Stelle im Dokument\]](#)

24 DHCP-Optionen 66 und 67: DHCP-Bootparameter: Option 66 verweist auf den Bootserver, Option 67 auf die zu ladende Startdatei. [Quelle: Microsoft PXE-Troubleshooting](#). [\[zurück zur Stelle im Dokument\]](#)

25 bootx64.efi: UEFI-Startdatei für 64-Bit-Systeme, die beim Netzwerkstart als Bootloader geladen werden kann. [Quelle: Microsoft PXE-Troubleshooting](#). [\[zurück zur Stelle im Dokument\]](#)